

Parallel Programming Patterns

McCool et al., Chapter 3

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

Copyright © 2013, 2017–2026
Moreno Marzolla, Università di Bologna, Italy
<https://www.moreno.marzolla.name/teaching/CP/>



Except where stated otherwise, this work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

What is a *pattern*?

- A design pattern is “*a general solution to a recurring engineering problem*”.
- A design pattern is **not** a ready-made solution to a given problem...
- ...rather, it is a description of how a certain **family** of problems can be solved.

Architectural patterns

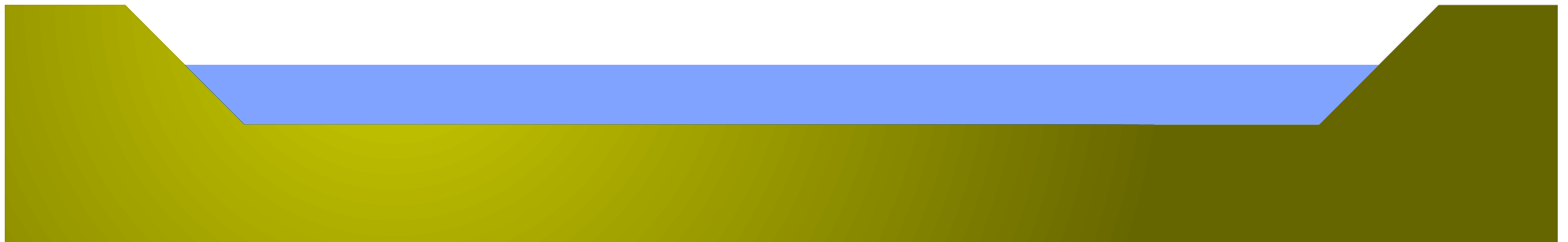
- The term “*architectural pattern*” was first used by architect Christopher Alexander to denote common design decision that have been used by architects and engineers to realize buildings and constructions in general.



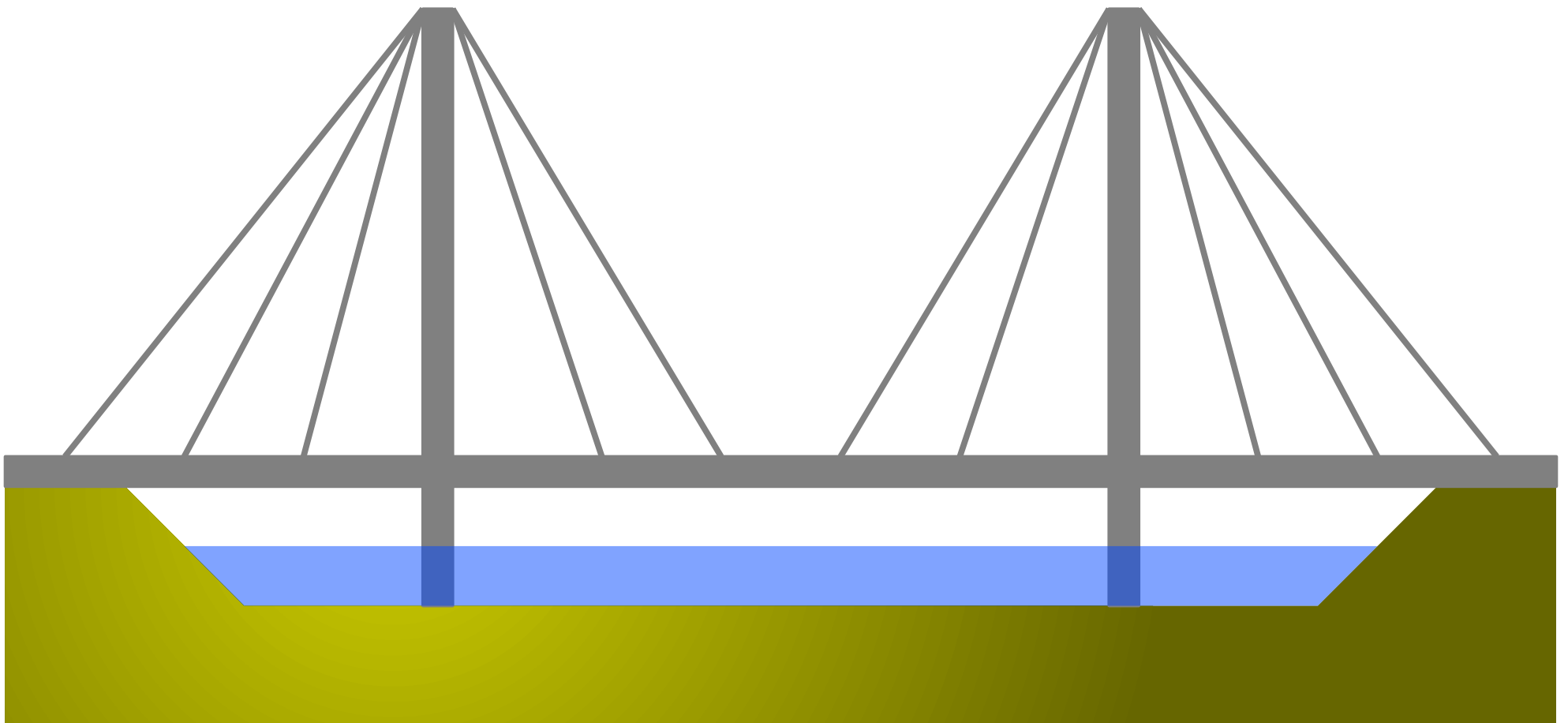
Christopher Alexander,
(1936--), *A Pattern Language:
Towns, Buildings, Construction*

Example

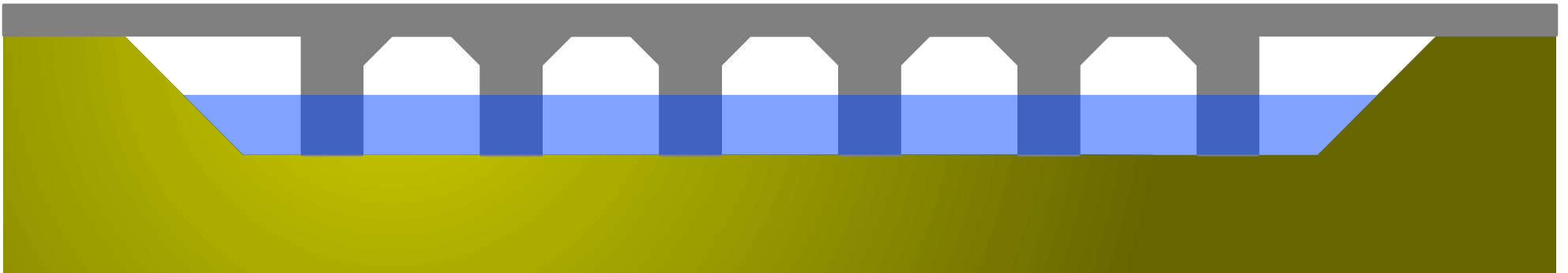
- Building a bridge across a river.
- You do not “invent” a new type of bridge each time.
 - Instead, you adapt an already existing type of bridge.



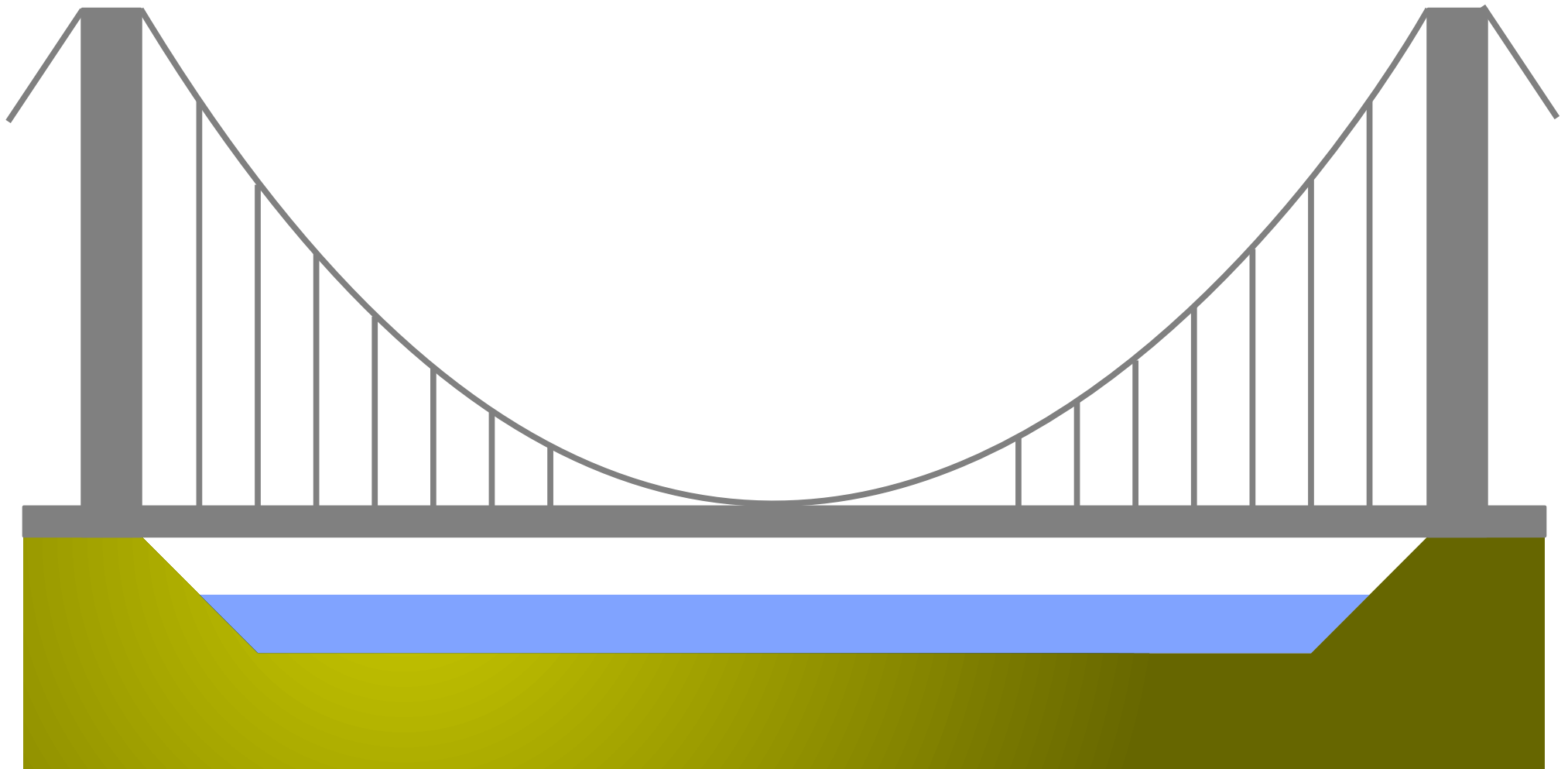
Example



Example



Example



Parallel programming patterns

- Writing parallel algorithms is **hard**.
 - For some problems, the parallel solution has very little (if anything) to do with the serial one.
- Fortunately, many problems can be solved by combining a small set of parallel programming patterns, some of which will be described in this lecture.

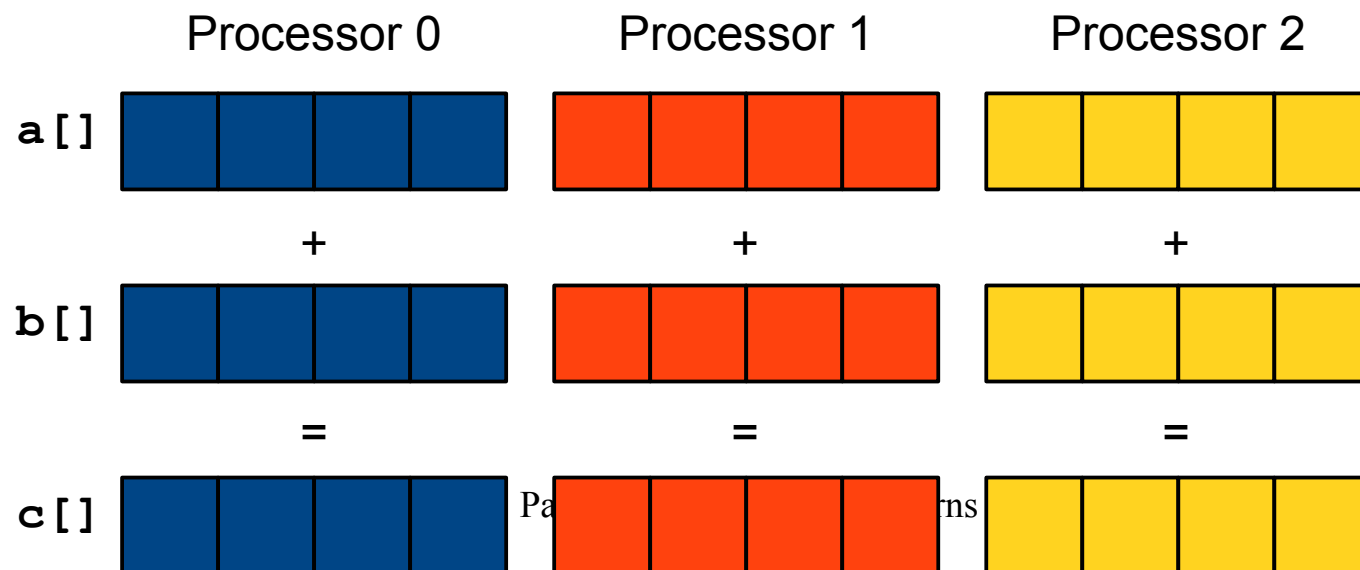
Some Parallel Programming Patterns

- Embarrassingly Parallel.
- Scatter/Gather.
- Partition.
- Master-Worker.
- Stencil.
- Reduce.
- Scan.
- List Ranking (Pointer Jumping).

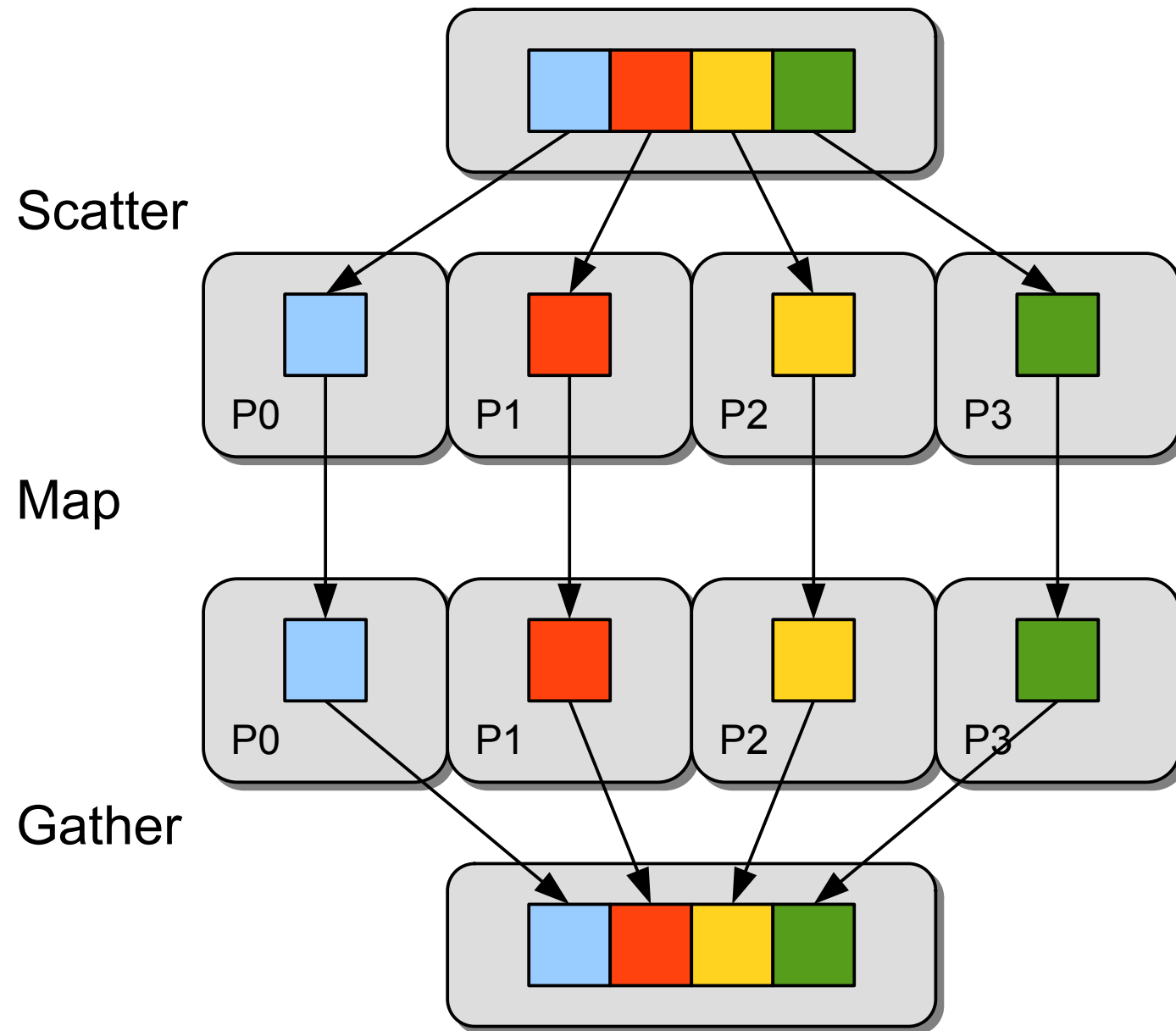
Embarrassingly parallel

Embarrassingly Parallel

- Applies when the computation can be decomposed in **independent** tasks that can be solved with little or no communication.
- Examples:
 - Vector sum.
 - Mandelbrot set.
 - 3D rendering.
 - Brute force password cracking.
 - ...



Scatter/Gather



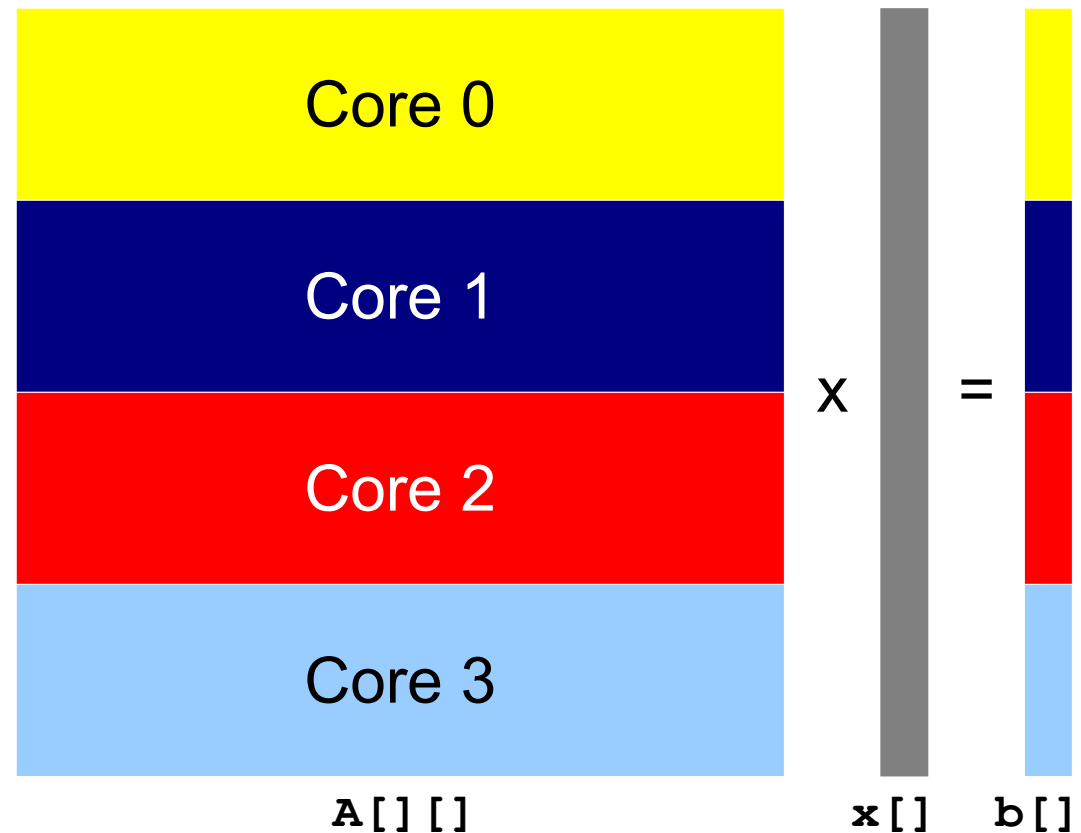
Partition

Partition

- The input data space (in short, *domain*) is split in disjoint regions called *partitions*
- Each processor operates on one partition
- This pattern is particularly useful when the application exhibits *locality of reference*
 - i.e., when processors can refer to their own partition only and need little or no communication with other processors

Example

- Matrix-vector product
 $Ax = b$
- Matrix $A[] []$ is partitioned into P horizontal blocks
- Each processor
 - operates on one block of $A[] []$ and on a full copy of $x[]$
 - computes a portion of the result $b[]$



Partition

- Types of partition:
 - *Regular*: the domain is split into partitions of roughly the same size and shape. E.g., matrix-vector product.
 - *Irregular*: partitions do not necessarily have the same size or shape. E.g., heat transfer on irregular solids.
- Size of partitions (*granularity*):
 - *Fine-Grained*: a large number of small partitions.
 - *Coarse-Grained*: a few large partitions.

1-D Partitioning

- Block

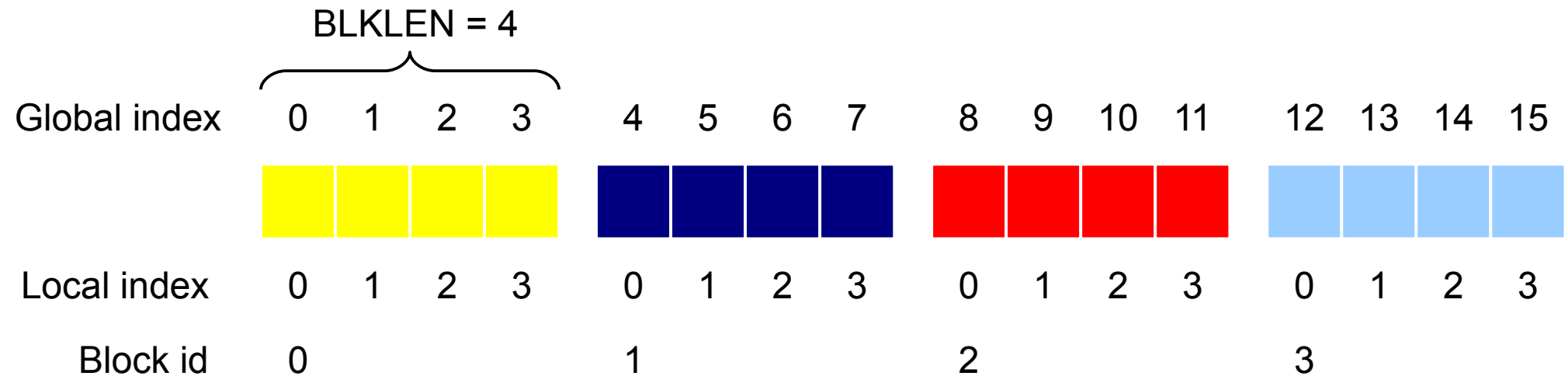


- Cyclic



Block partitioning

- Each element has a **global** index and a **local** index

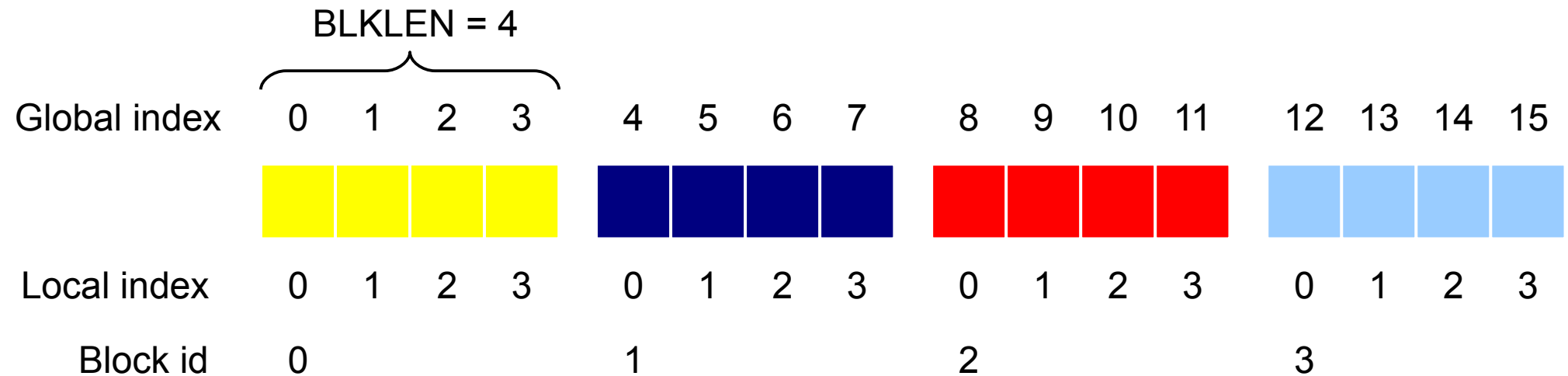


- If all blocks have the same length *BLKLEN*

$$global_index = local_index + block_id \times BLKLEN$$

Block partitioning

- Each element has a **global** index and a **local** index



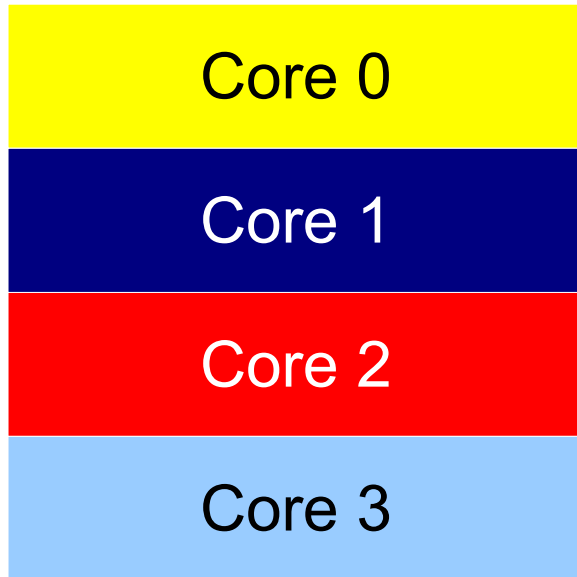
- If all blocks have the same length *BLKLEN*

*Integer division,
as in the C
language*

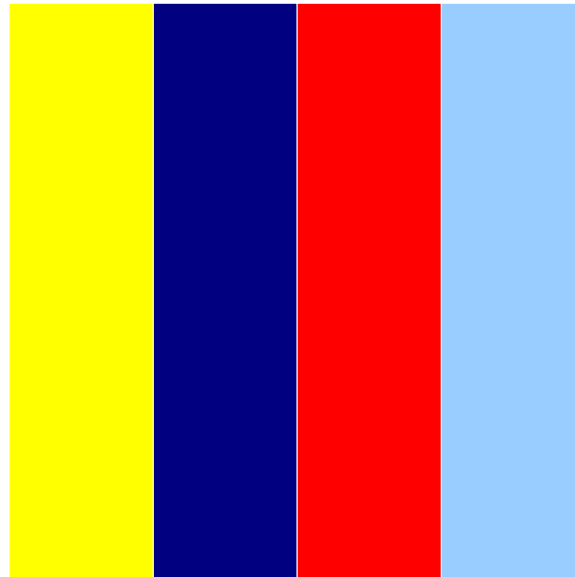
$$\begin{aligned} \text{block_id} &= \text{global_index} / \text{BLKLEN} \\ \text{local_index} &= \text{global_index} \% \text{BLKLEN} \end{aligned}$$

2-D Block Partitioning

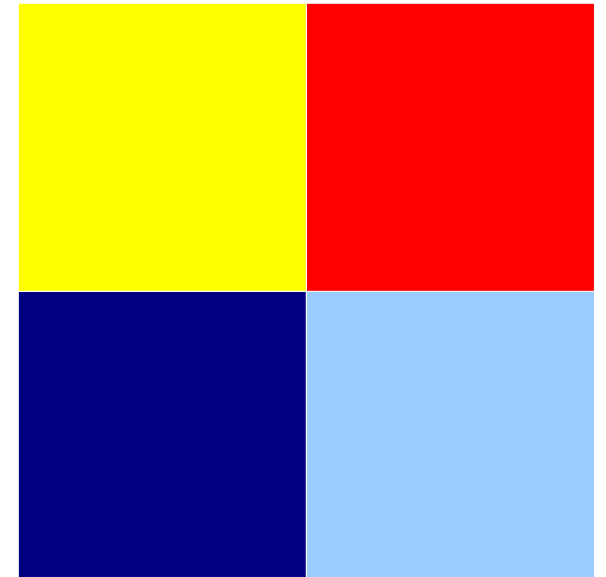
Block, *



*, Block

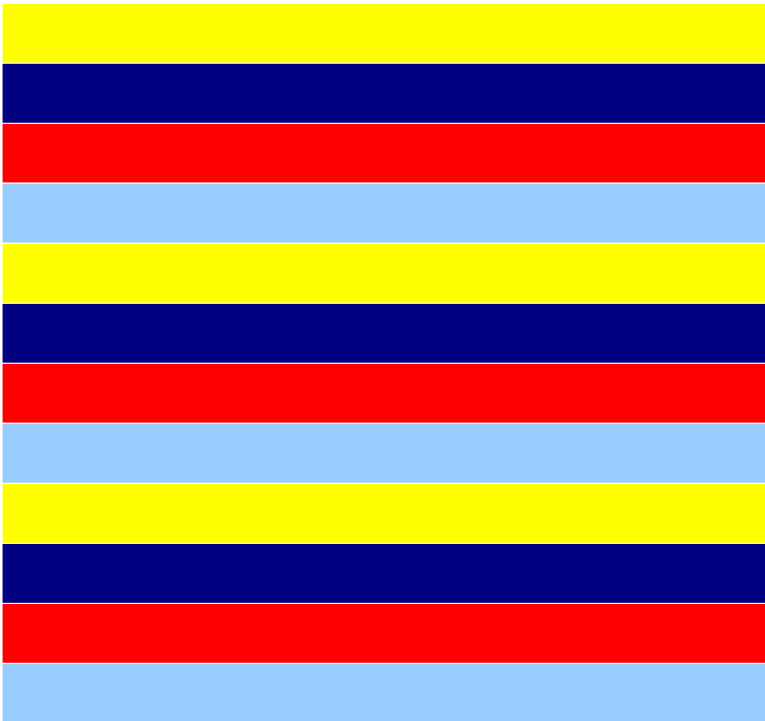


Block, Block

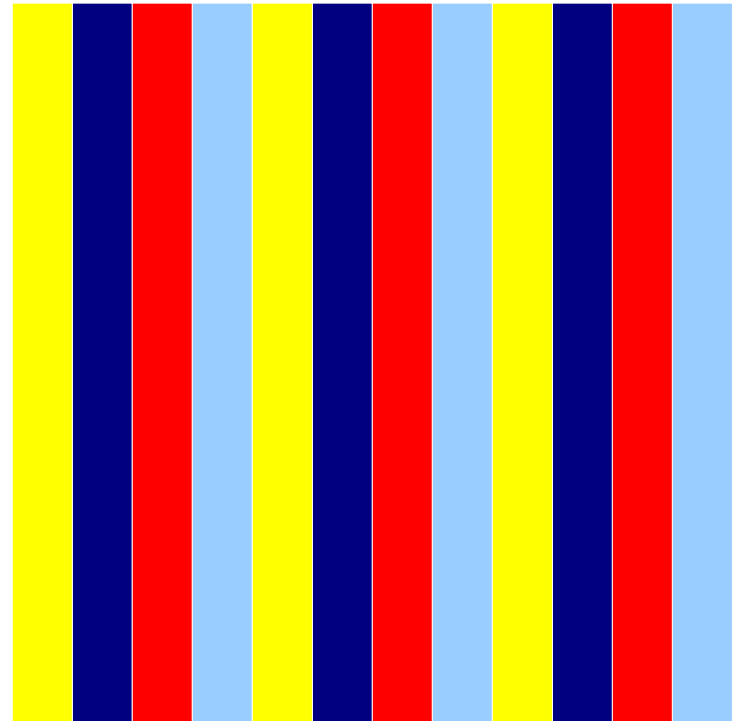


2-D Cyclic Partitioning

Cyclic, *

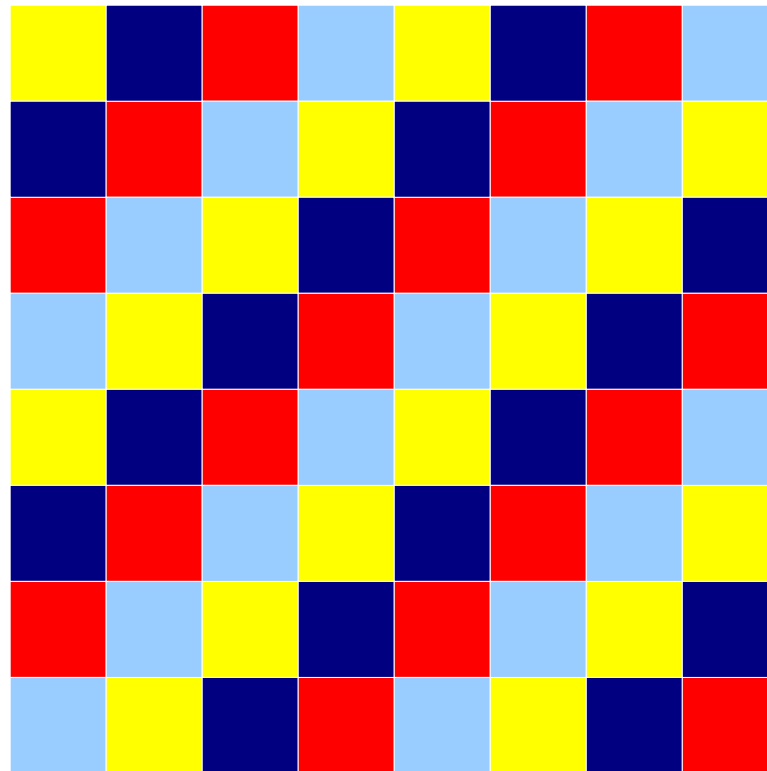


*, Cyclic



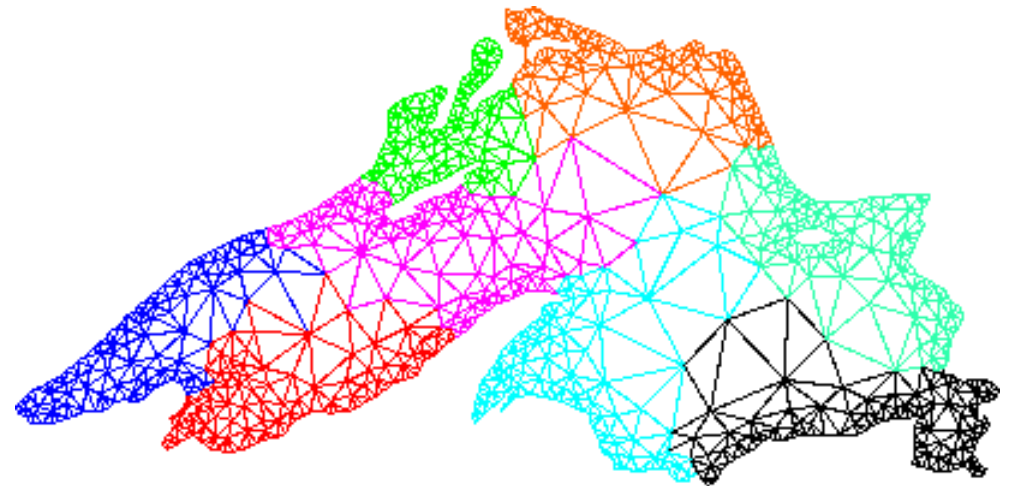
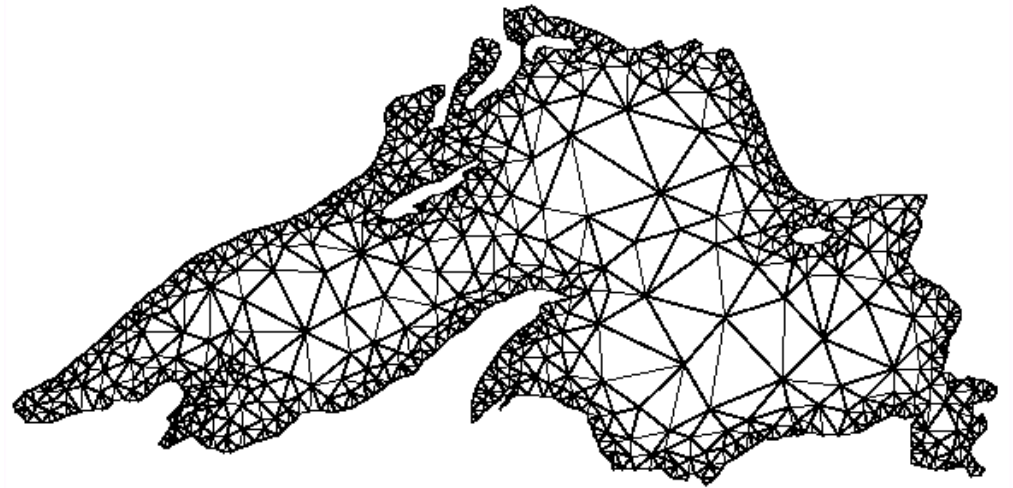
2-D Cyclic Partitioning

Cyclic-cyclic



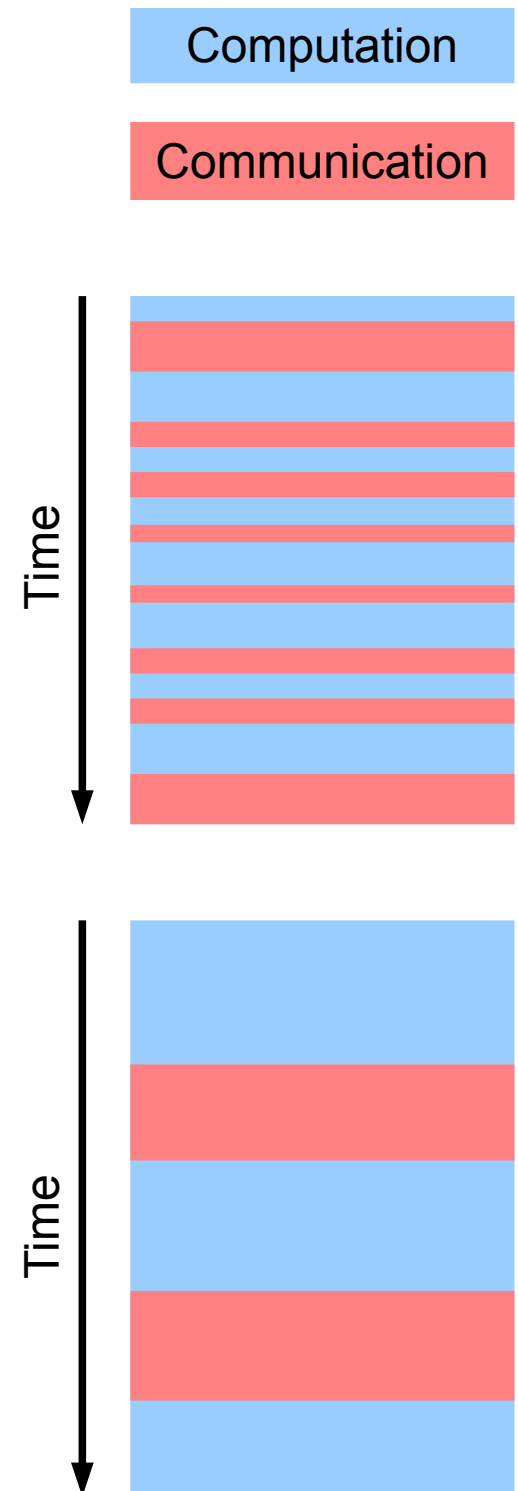
Irregular partitioning example

- A lake surface is approximated with a triangular mesh.
- Colors indicate the mapping of mesh elements to processors.



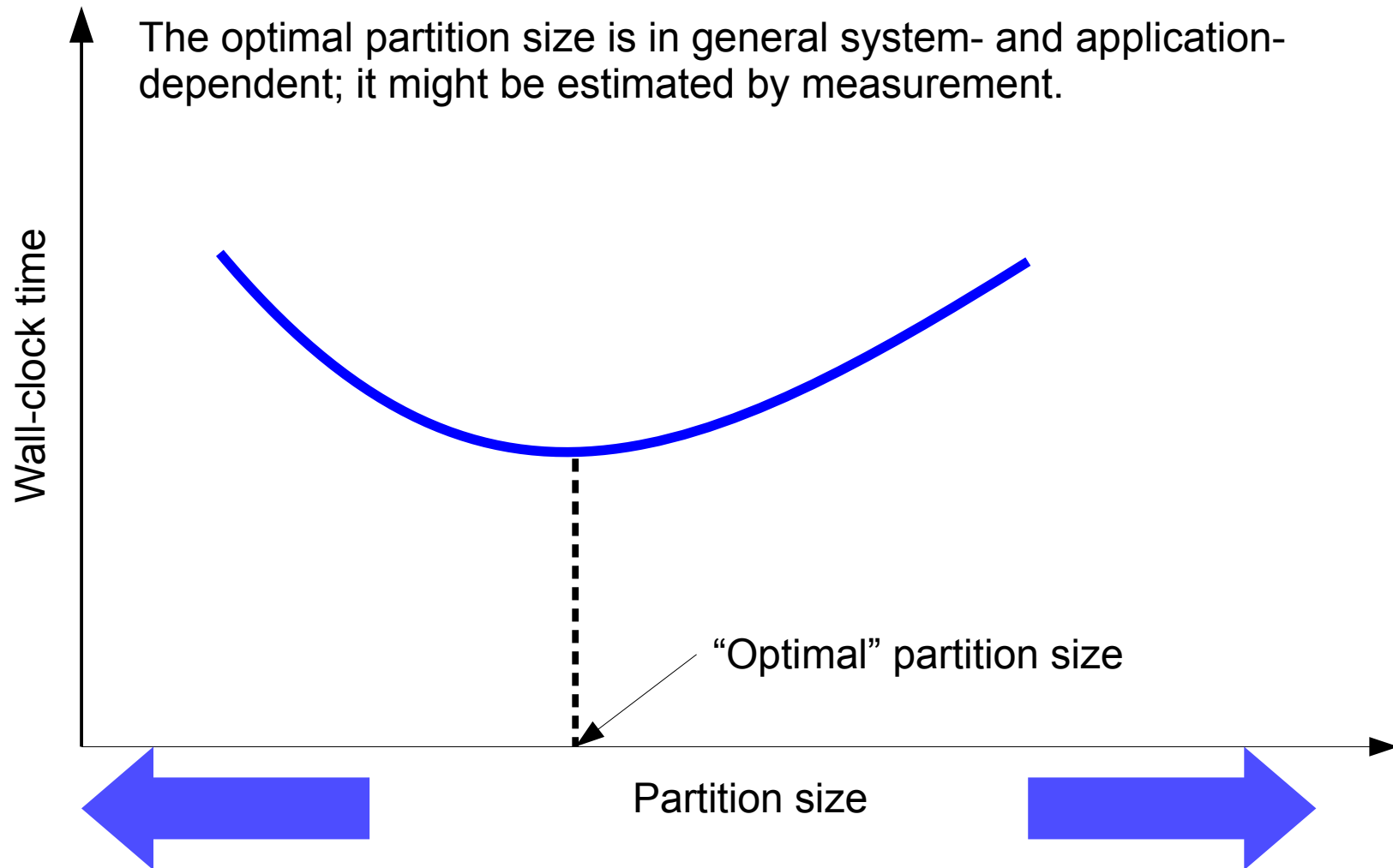
Fine grained vs Coarse grained partitioning

- *Fine-grained Partitioning*
 - Better load balancing, especially if combined with the master-worker pattern (see later).
 - If granularity is too fine, the computation / communication ratio might become too low (communication dominates on computation).
- *Coarse-grained Partitioning*
 - In general improves the computation / communication ratio.
 - However, it might cause load imbalance.
- The "optimal" granularity is sometimes problem-dependent; in other cases the user must choose which granularity to use.



Choosing the partition size

The optimal partition size is in general system- and application-dependent; it might be estimated by measurement.



Too small = higher scheduling overhead

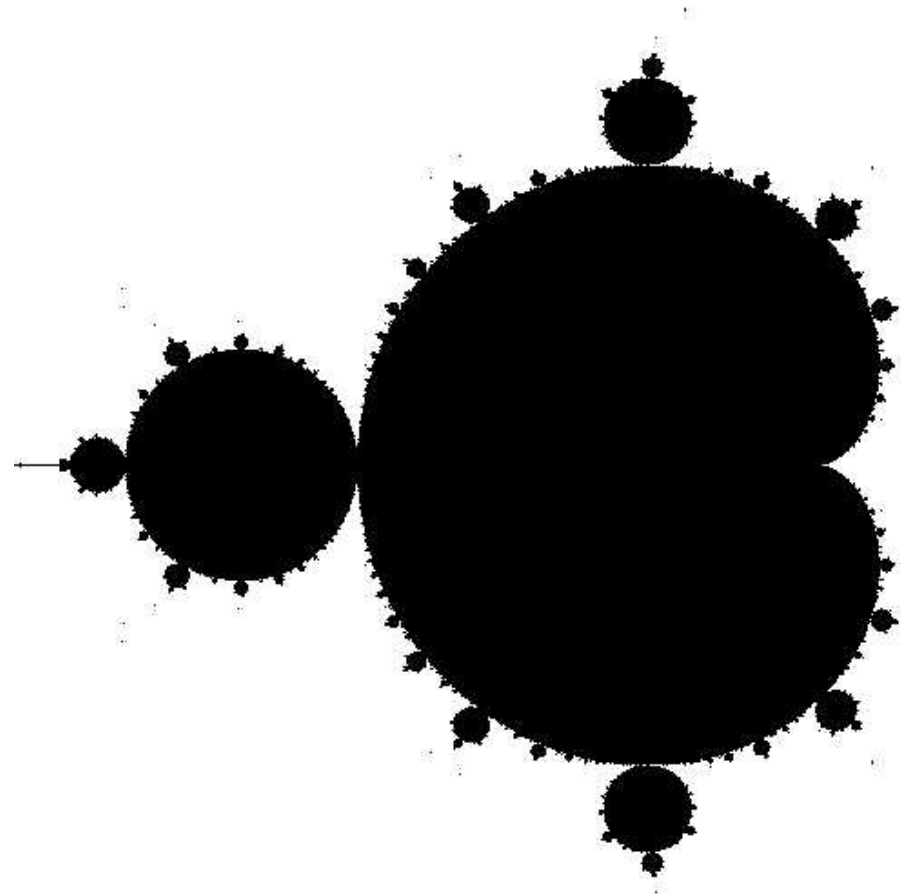
Too large = unbalanced workload

Example: Mandelbrot set

- The Mandelbrot set is the set of points c on the complex plane s.t. the sequence $z_n(c)$ defined as

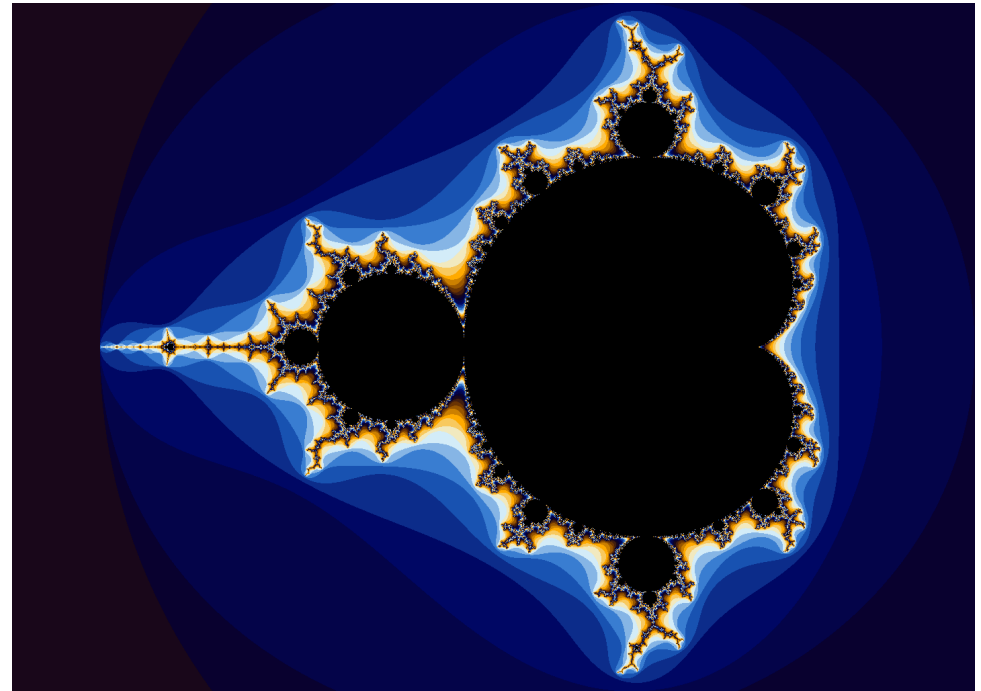
$$z_n(c) = \begin{cases} 0 & \text{if } n=0 \\ z_{n-1}^2(c) + c & \text{otherwise} \end{cases}$$

does not diverge when $n \rightarrow +\infty$.



Mandelbrot set in color

- If the modulus of $z_n(c)$ does not exceed 2 after *maxit* iterations, the pixel is black
 - the point is assumed to be part of the Mandelbrot set.
- Otherwise, the color depends on the number of iterations required for the modulus of $z_n(c)$ to become > 2 .



Pseudocode

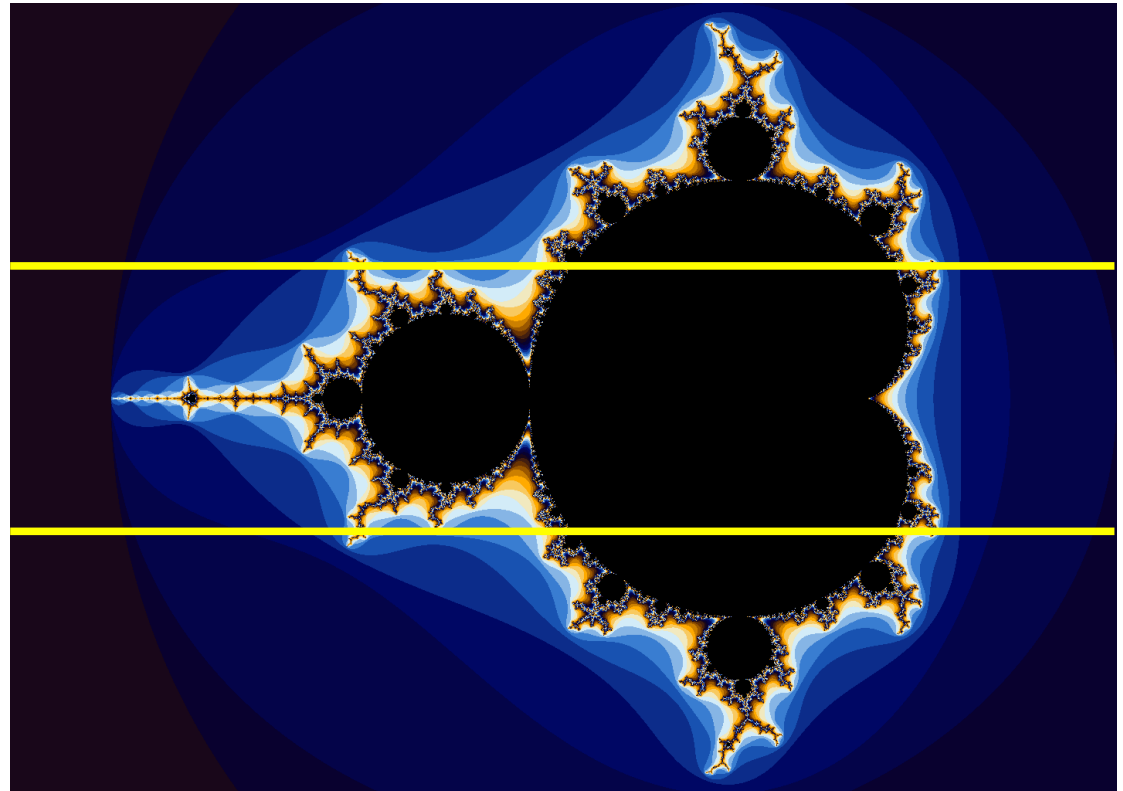
Embarrassingly parallel structure: the color of each pixel can be computed independently from other pixels

```
maxit = 1000;
for each point (cx, cy) {
    x = y = 0;
    it = 0;
    while ( it < maxit AND  $x*x + y*y \leq 2*2$  ) {
        xnew = x*x - y*y + cx;
        ynew = 2*x*y + cy;
        x = xnew;
        y = ynew;
        it = it + 1;
    }
    plot(cx, cy, it);
}
```

Source: http://en.wikipedia.org/wiki/Mandelbrot_set#For_programmers

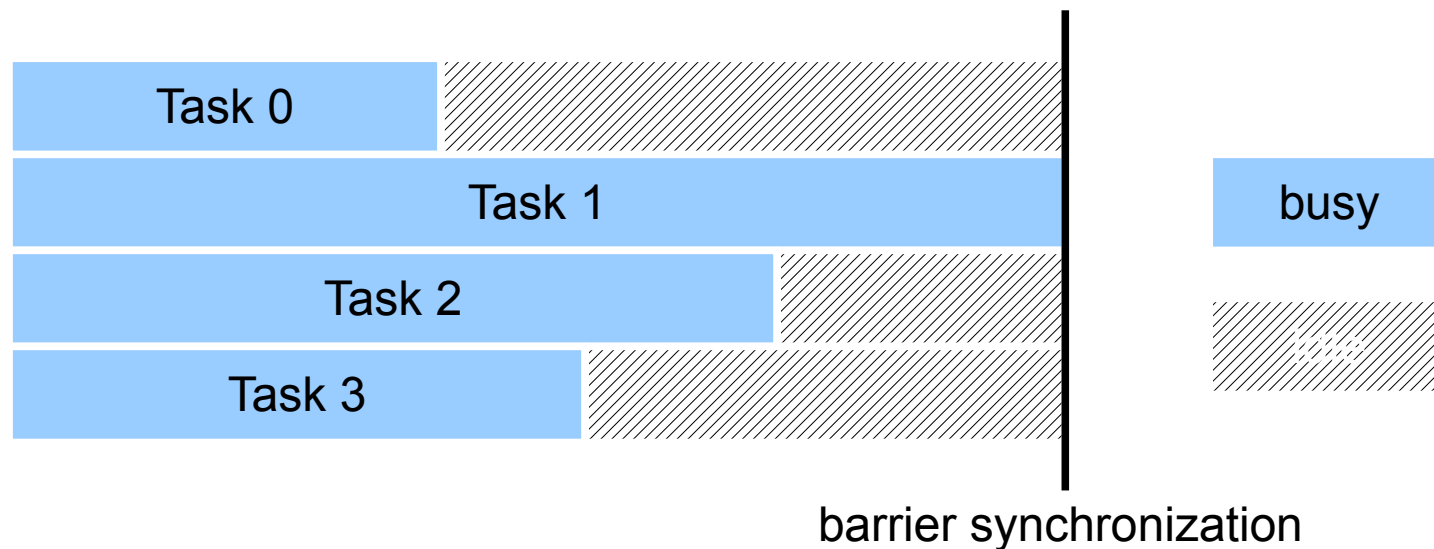
Mandelbrot set

- A regular partitioning can result in uneven load distribution:
 - Black pixels require *maxit* iterations.
 - Other pixels require fewer iterations.



Load balancing

- Ideally, each processor should perform the same amount of work.
 - If the tasks synchronize at the end of the computation, the execution time will be that of the **slower** task.



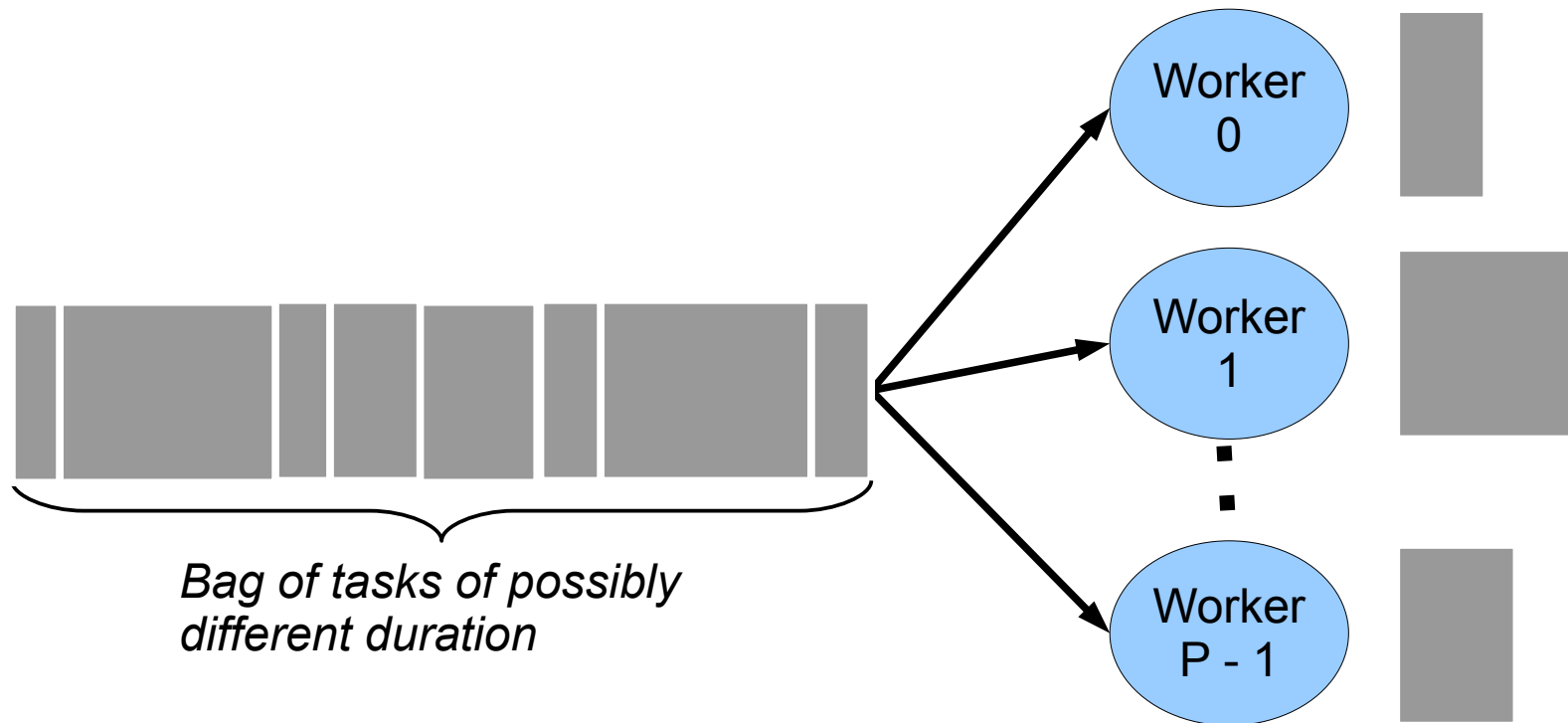
Load balancing HowTo

- The workload is balanced if each processor performs more or less the same amount of work.
- Ways to achieve load balancing:
 - Use fine-grained partitioning
 - ...but beware of the possible communication overhead if the tasks need to communicate.
 - Use dynamic task allocation (**master-worker** paradigm)
 - ...but beware that dynamic task allocation might incur in higher overhead with respect to static task allocation.

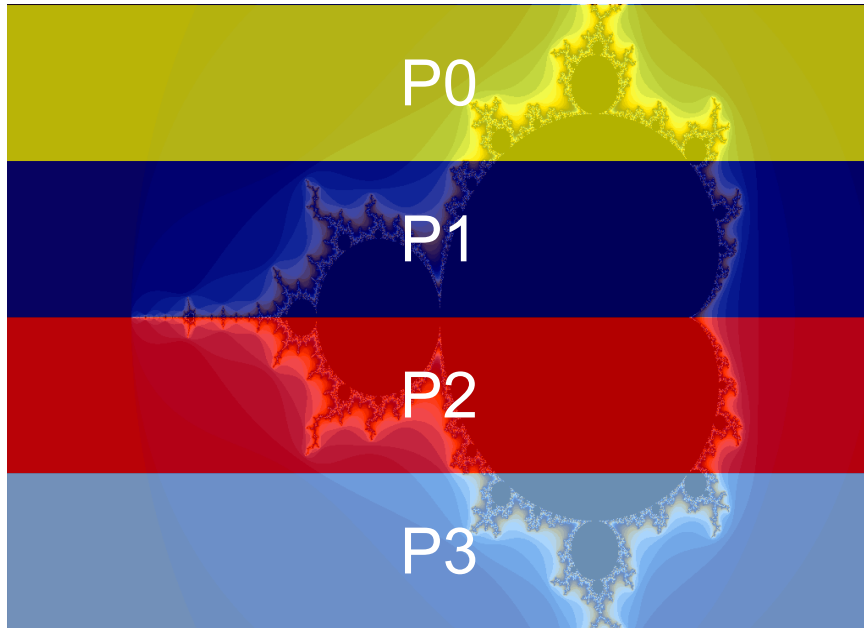
Master-worker paradigm

(process farm, work pool)

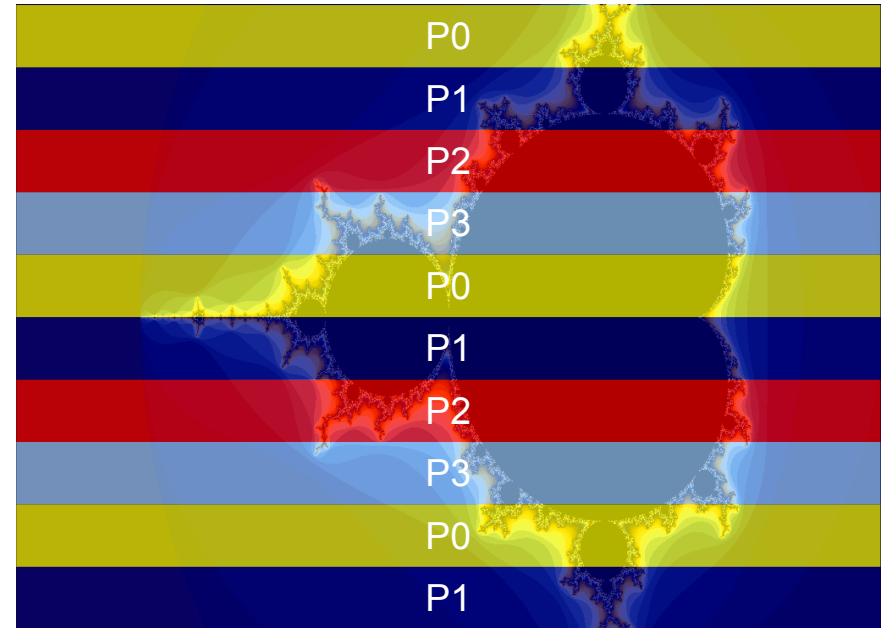
- Usually requires fine-grained partitioning
 - *number of task $\gg$ number of execution units*
- Each task is assigned to the first available worker



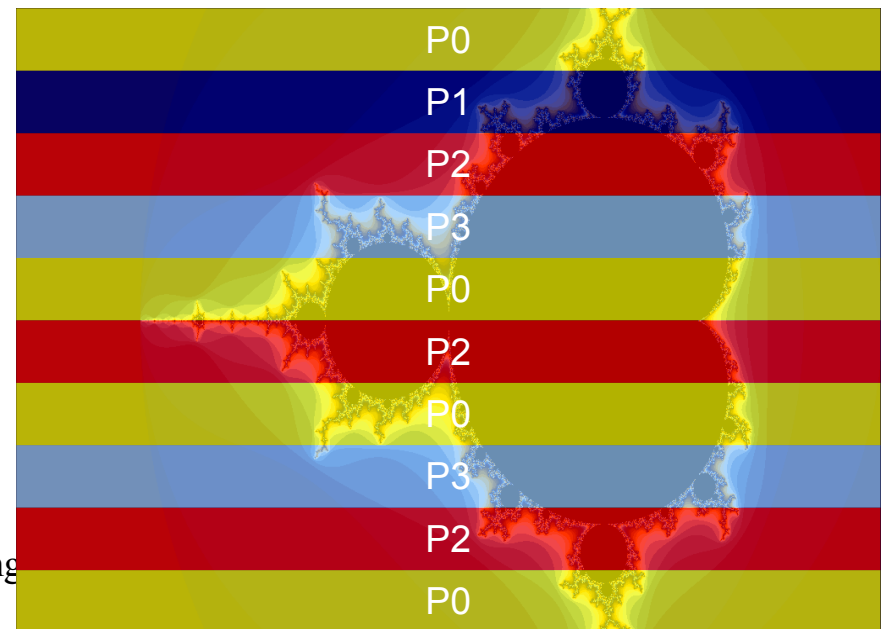
coarse-grained decomposition
static task assignment



block size = 64
static task assignment



block size = 64
dynamic (master-worker) task assignment
(example)



Example

omp-mandelbrot.c

- Coarse-grained partitioning
 - `OMP_SCHEDULE="static" ./omp-mandelbrot`
- Cyclic, fine-grained partitioning (64 rows per block)
 - `OMP_SCHEDULE="static,64" ./omp-mandelbrot`
- Dynamic, fine-grained partitioning (64 rows per block)
 - `OMP_SCHEDULE="dynamic,64" ./omp-mandelbrot`
- Dynamic, fine-grained partitioning (1 row per block)
 - `OMP_SCHEDULE="dynamic" ./omp-mandelbrot`

Stencil

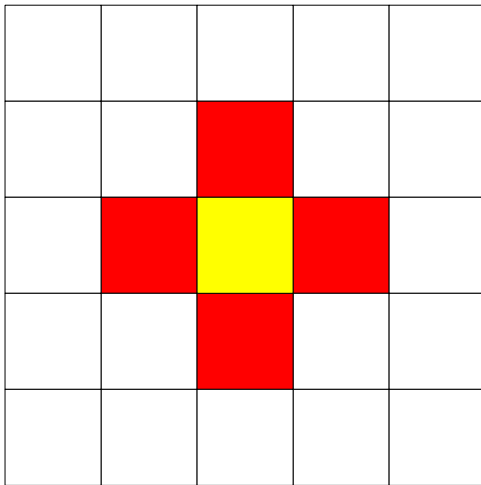
Stencil

- Stencil computations involve a grid whose values are updated according to a fixed pattern called *stencil*
 - Example: the Gaussian smoothing of an image updates the color of each pixel with the weighted average of the previous colors of the 5×5 neighborhood

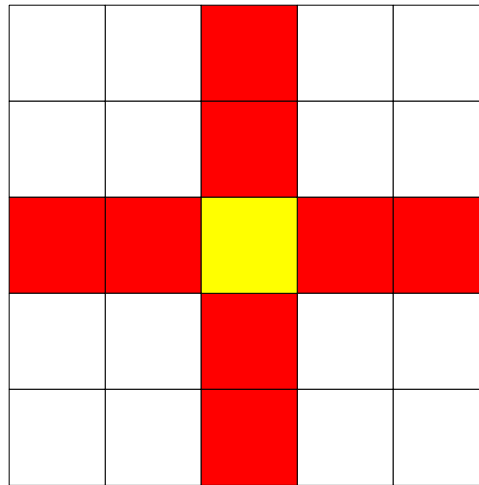
	1	4	5	4	1	
	4	16	28	16	4	
	7	28	41	28	7	
	4	16	28	16	4	
	1	4	7	4	1	

2D Stencils

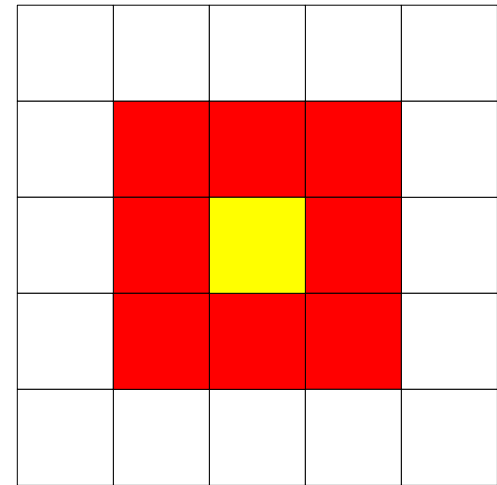
5-point 2-axis 2D stencil
(von Neumann neighborhood)



9-point 2-axis 2D stencil

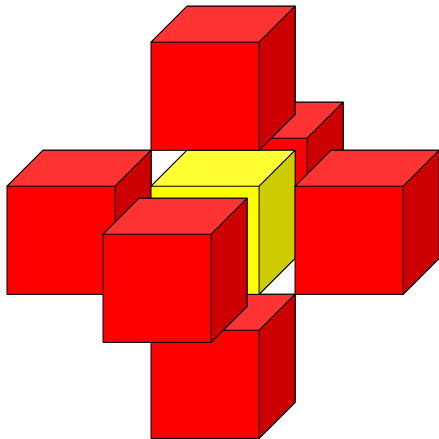


9-point 1-plane 2D stencil
(Moore neighborhood)

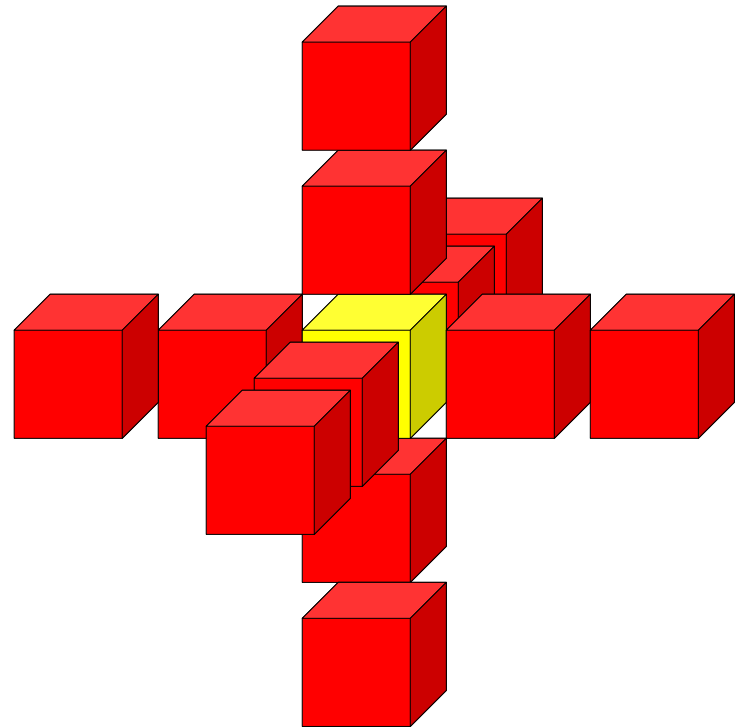


3D Stencils

7-point 3-axis 3D stencil

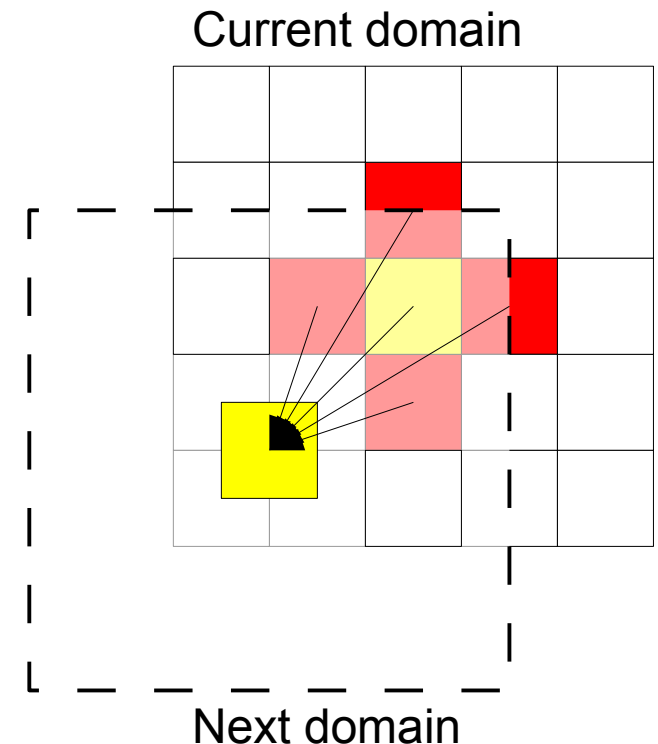


13-point 3-axis 3D stencil



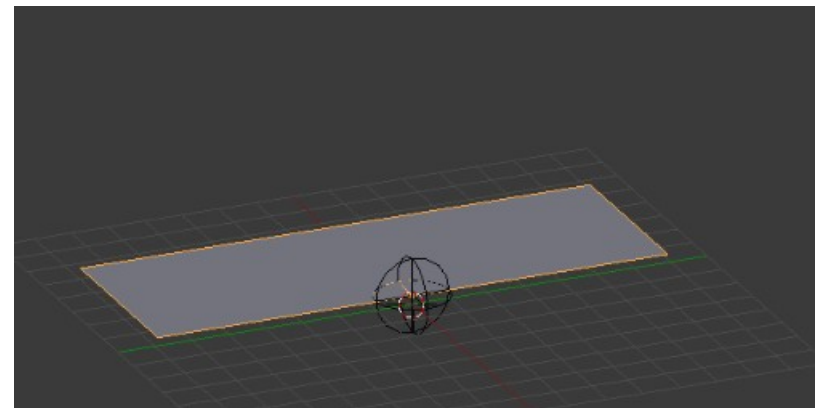
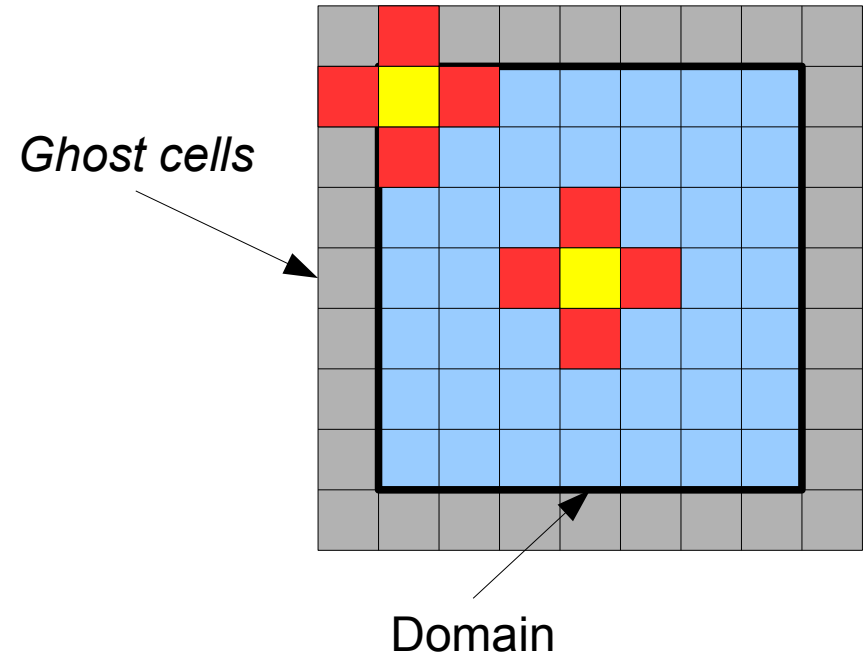
Synchronous updates

- Stencil computations usually require **synchronous updates**.
 - Assume one execution unit per cell.
 - Each unit accesses the neighborhood at the same time, computes the new value and stores it into a local register.
 - Then, each unit writes the new value into the domain at the same time.
- Most existing parallel architectures are **not** synchronous.
- However, we can simulate synchronous updates using **two** domains.
 - Values are **read** from the **current** domain.
 - New values are **written** to the **next** domain.
 - **current** and **next** are exchanged at the end of each step.

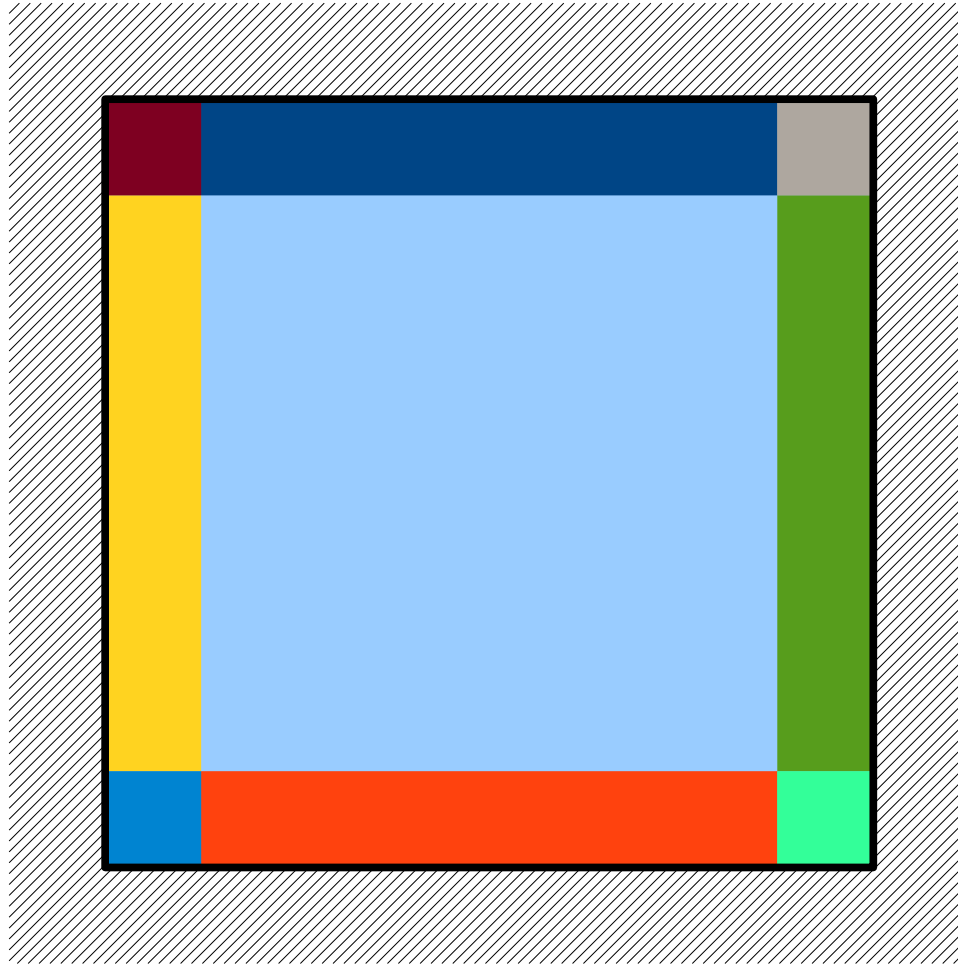


Ghost Cells

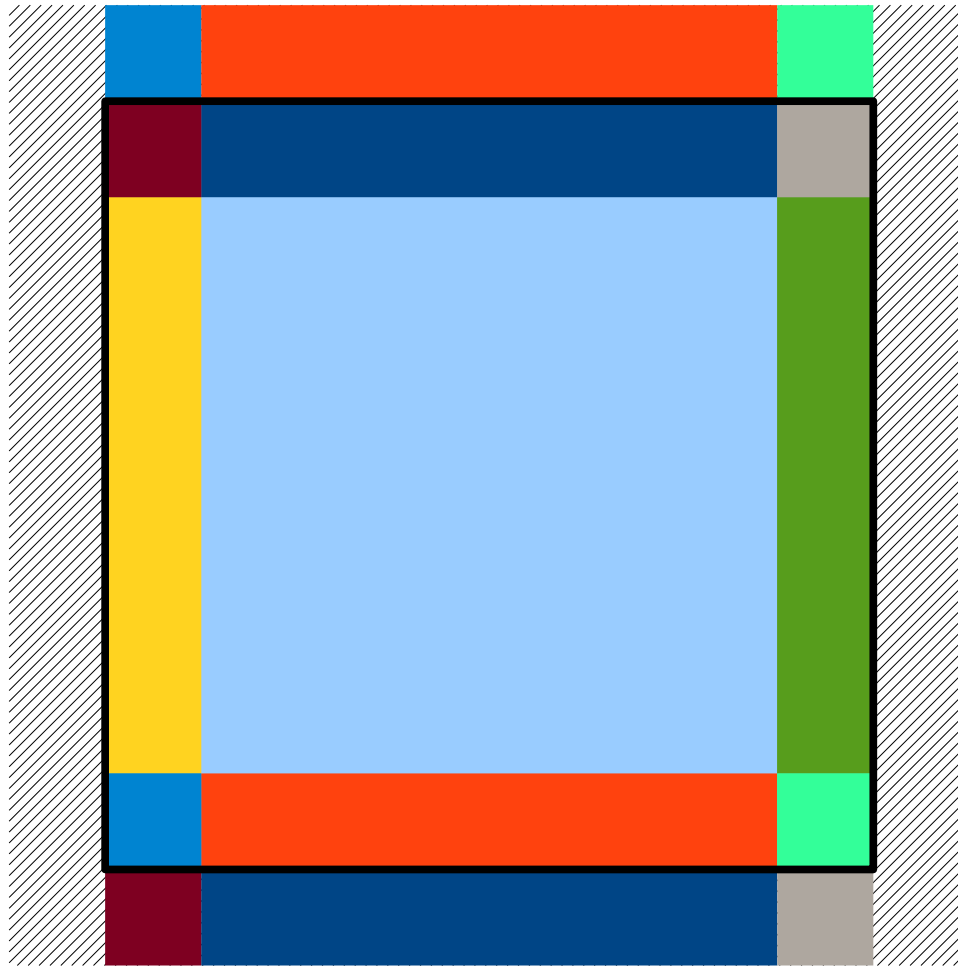
- How do we handle cells on the border?
 - For some applications, cells outside the domain must have some fixed value.
 - In other cases, we may assume **periodic boundary conditions**.
- In either case, we can extend the domain with **ghost cells**, so that cells on the border do not require any special treatment.



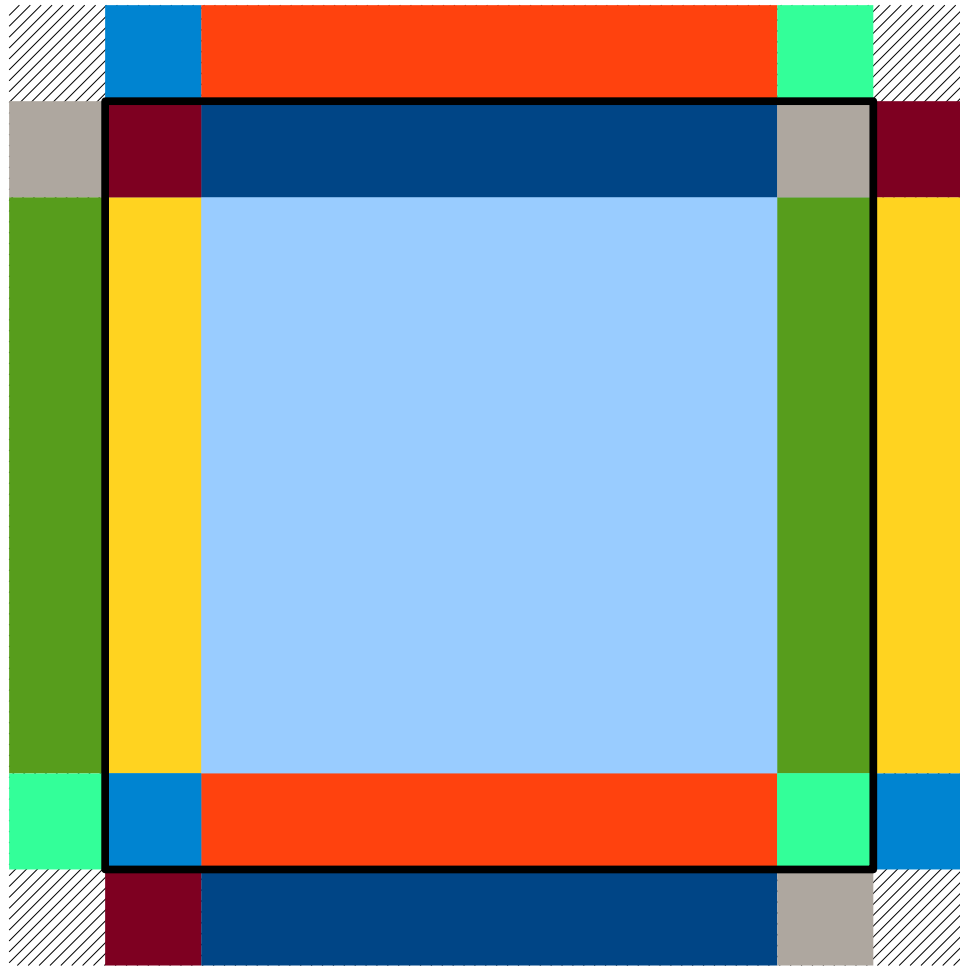
Periodic boundary conditions: How to fill ghost cells



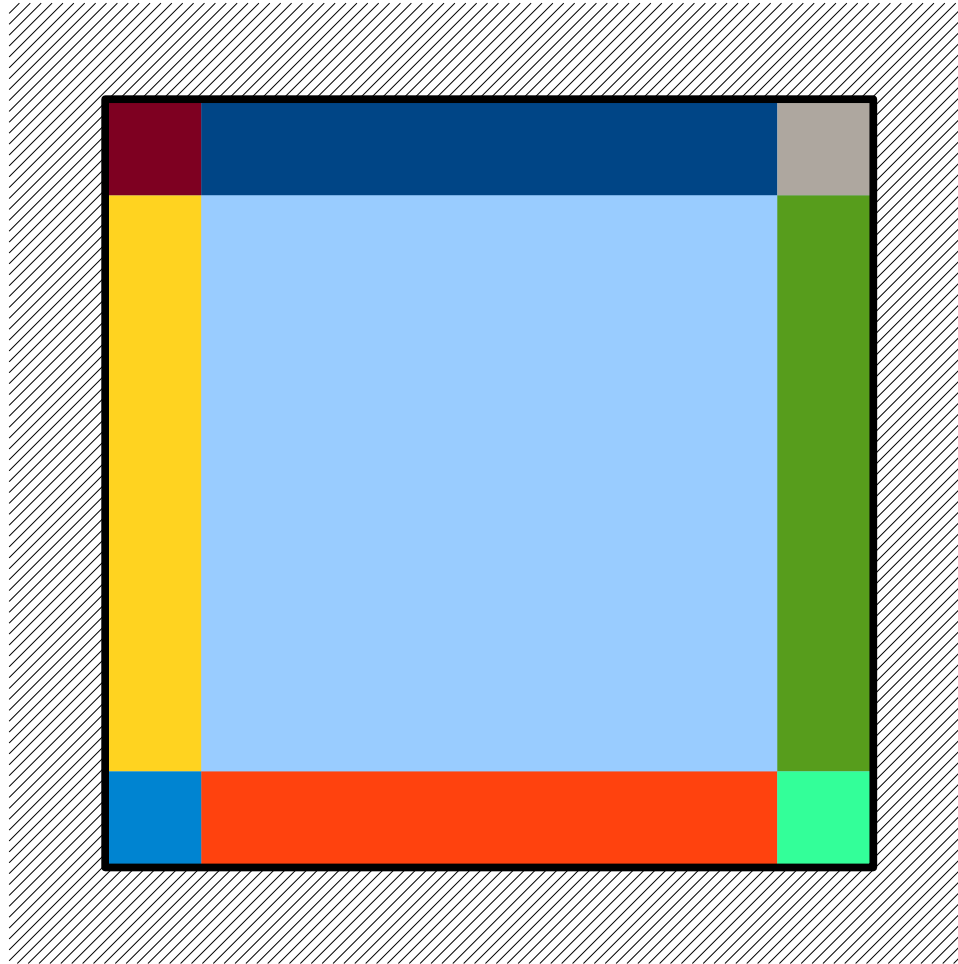
Periodic boundary conditions: How to fill ghost cells



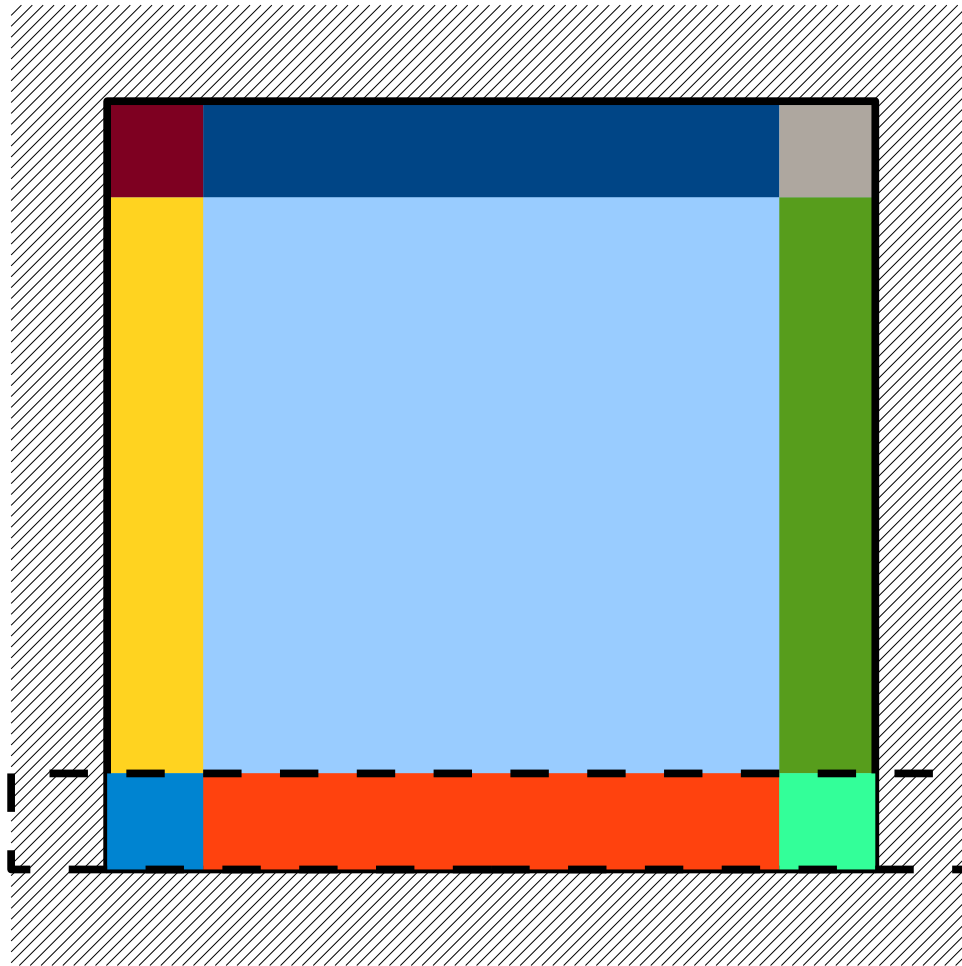
Periodic boundary conditions: How to fill ghost cells



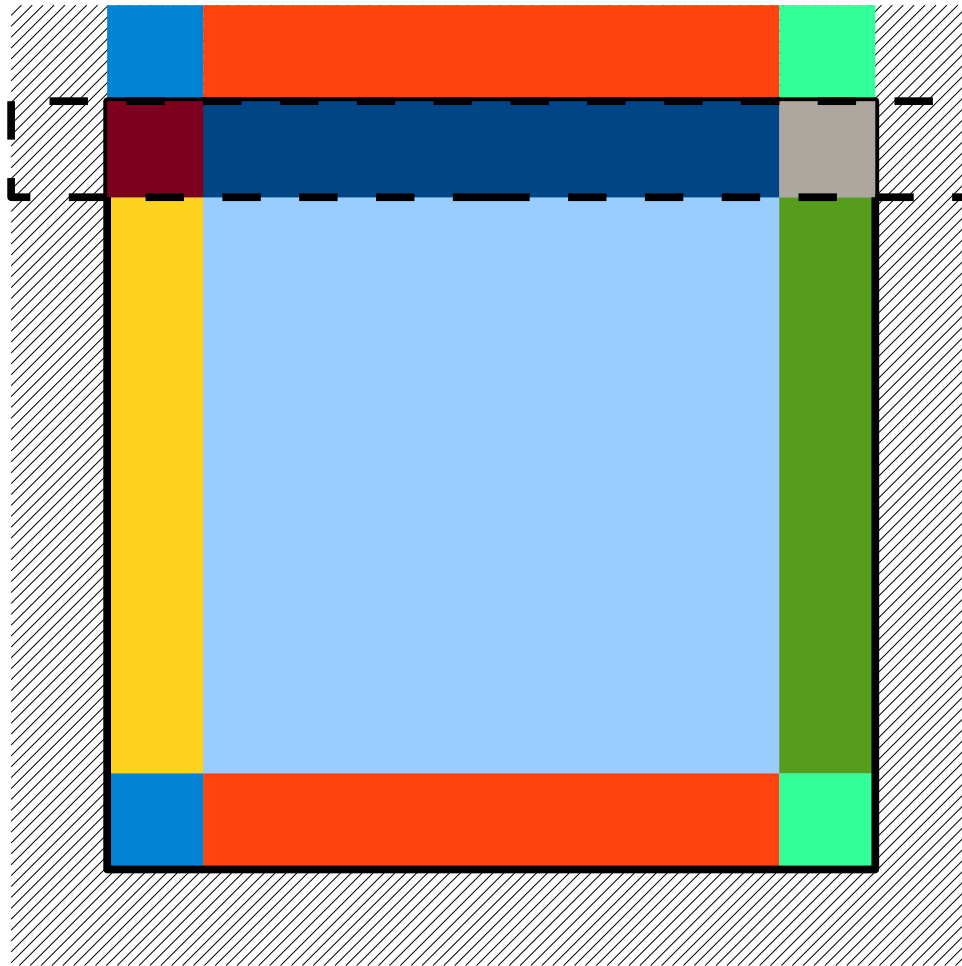
Periodic boundary conditions: Another way to fill ghost cells



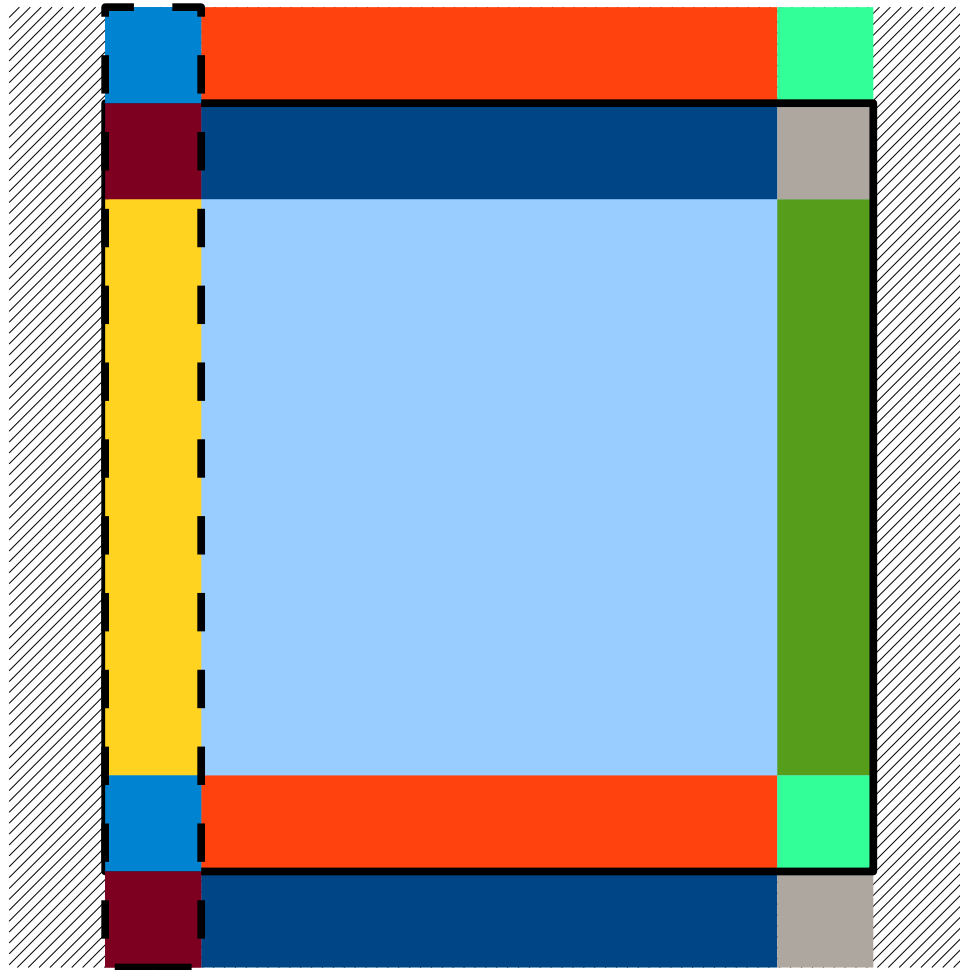
Periodic boundary conditions: Another way to fill ghost cells



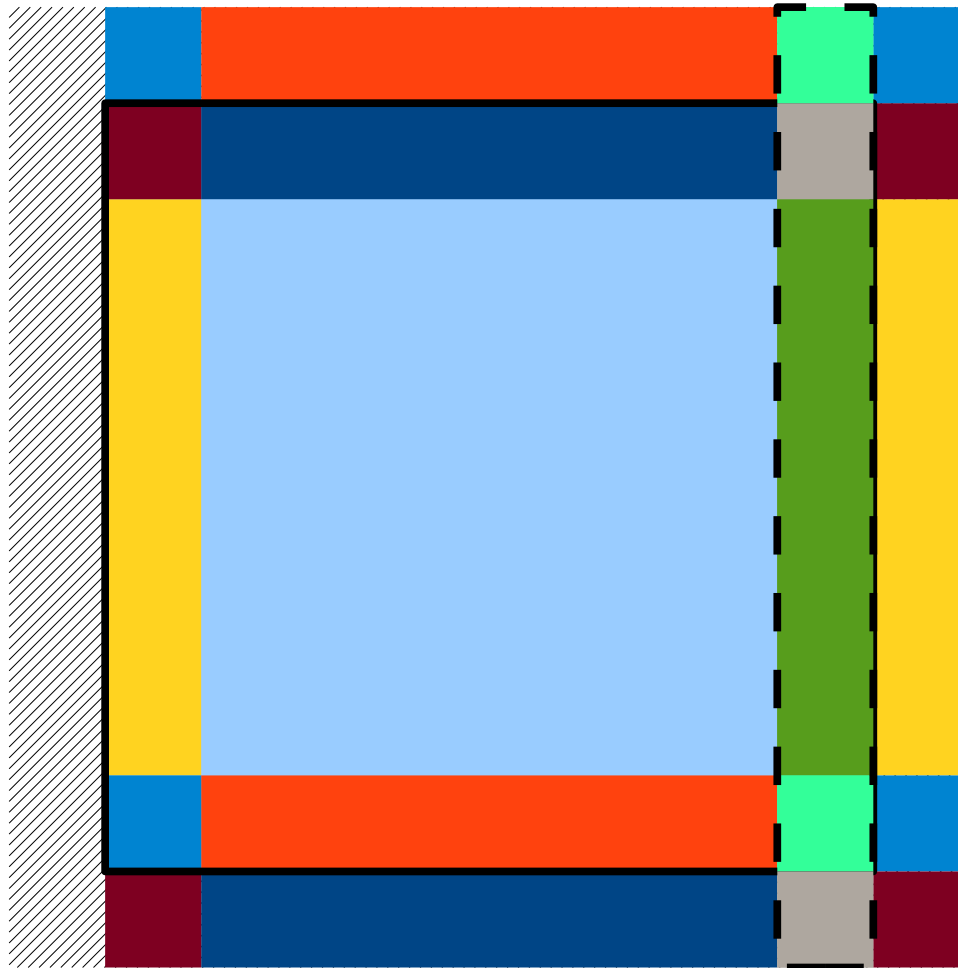
Periodic boundary conditions: Another way to fill ghost cells



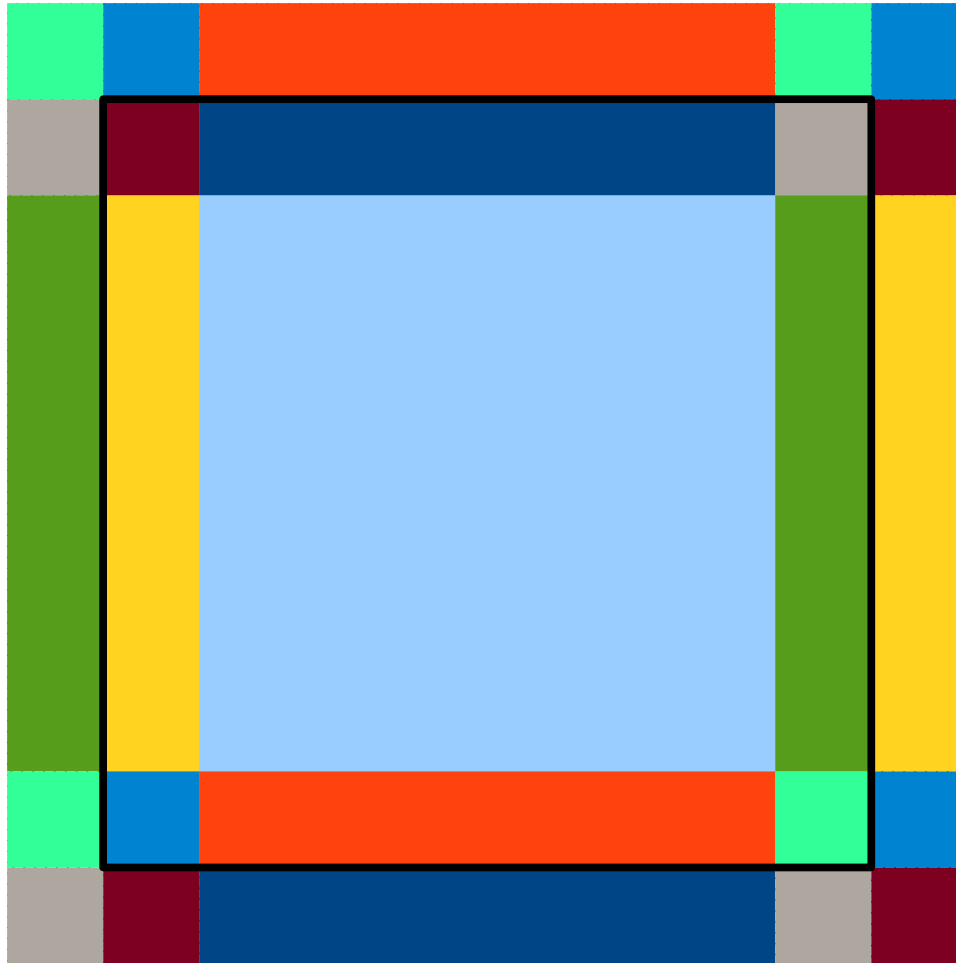
Periodic boundary conditions: Another way to fill ghost cells



Periodic boundary conditions: Another way to fill ghost cells



Periodic boundary conditions: Another way to fill ghost cells



Parallelizing stencil computations

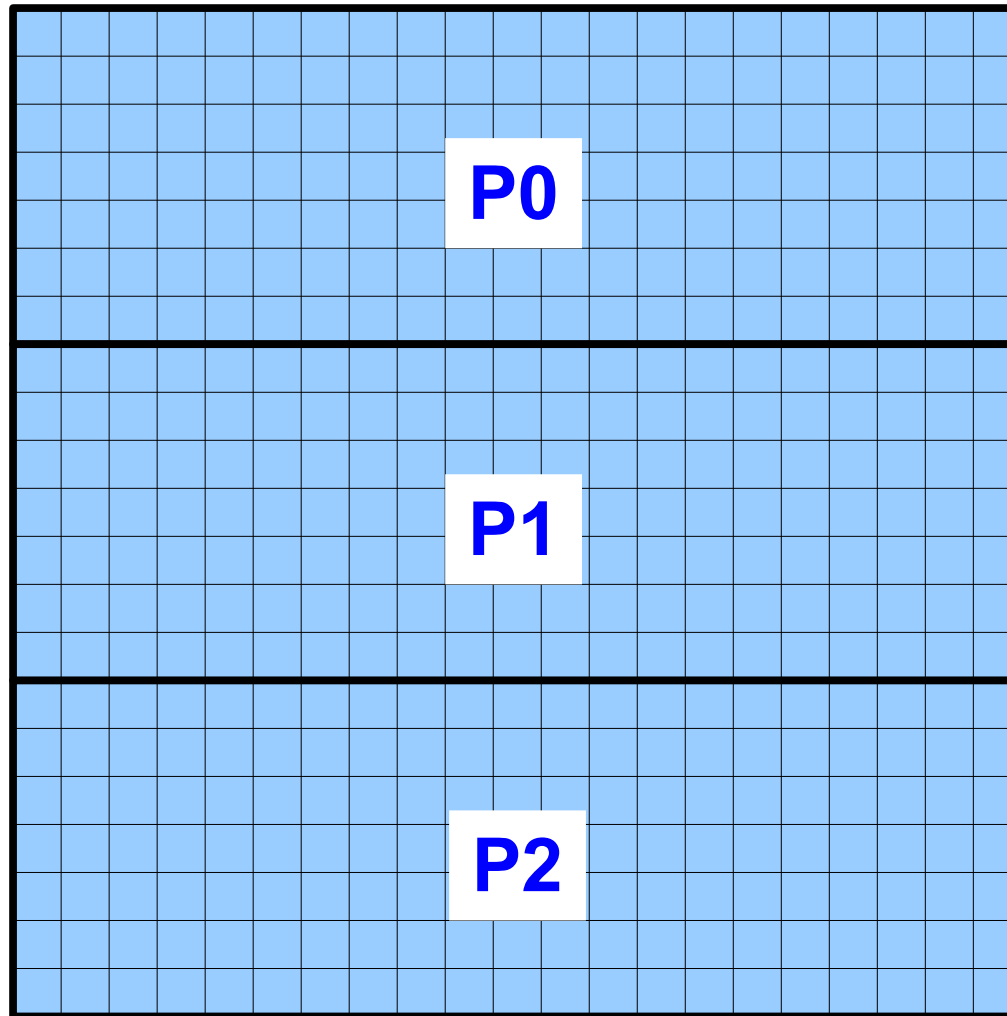
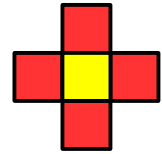
- Computing the next domain from the current one has embarrassingly parallel structure

```
Initialize current domain  
while (!terminated) {  
    Init ghost cells  
    Compute next domain in parallel  
    Exchange current and next domains  
}
```

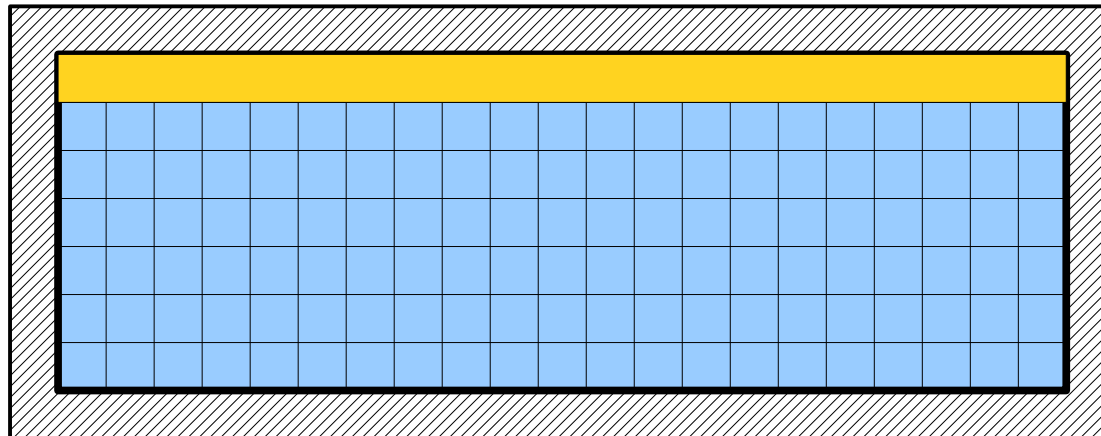
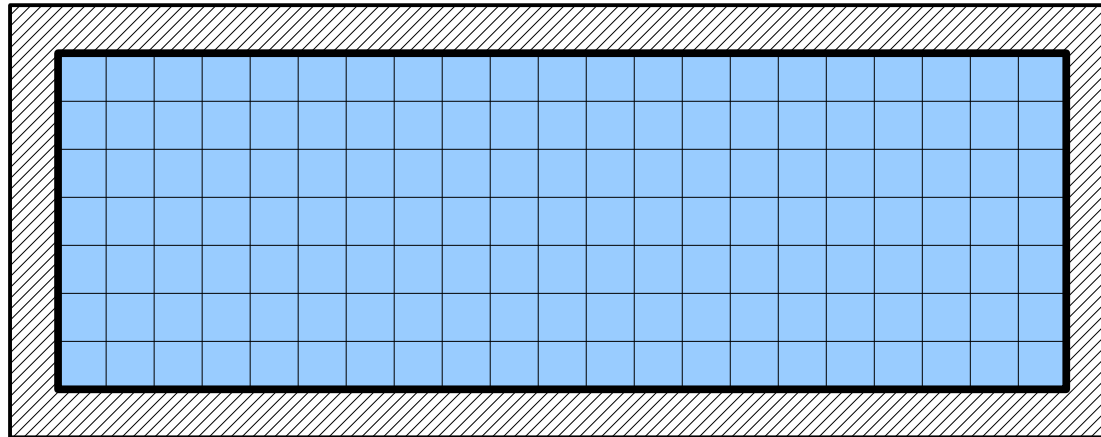
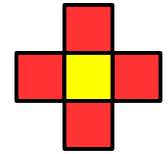
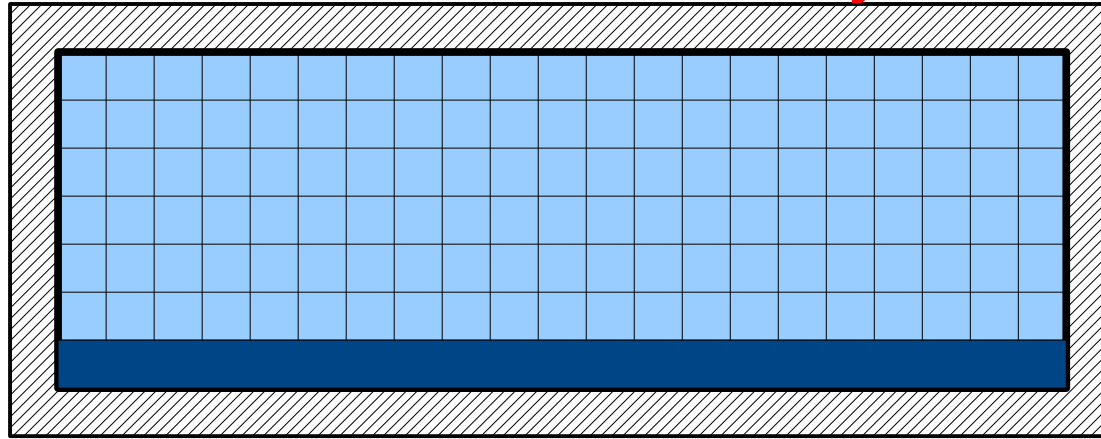
Stencil computations on distributed-memory architectures

- Ghost cells are essential to implement stencil computations efficiently on distributed-memory architectures.

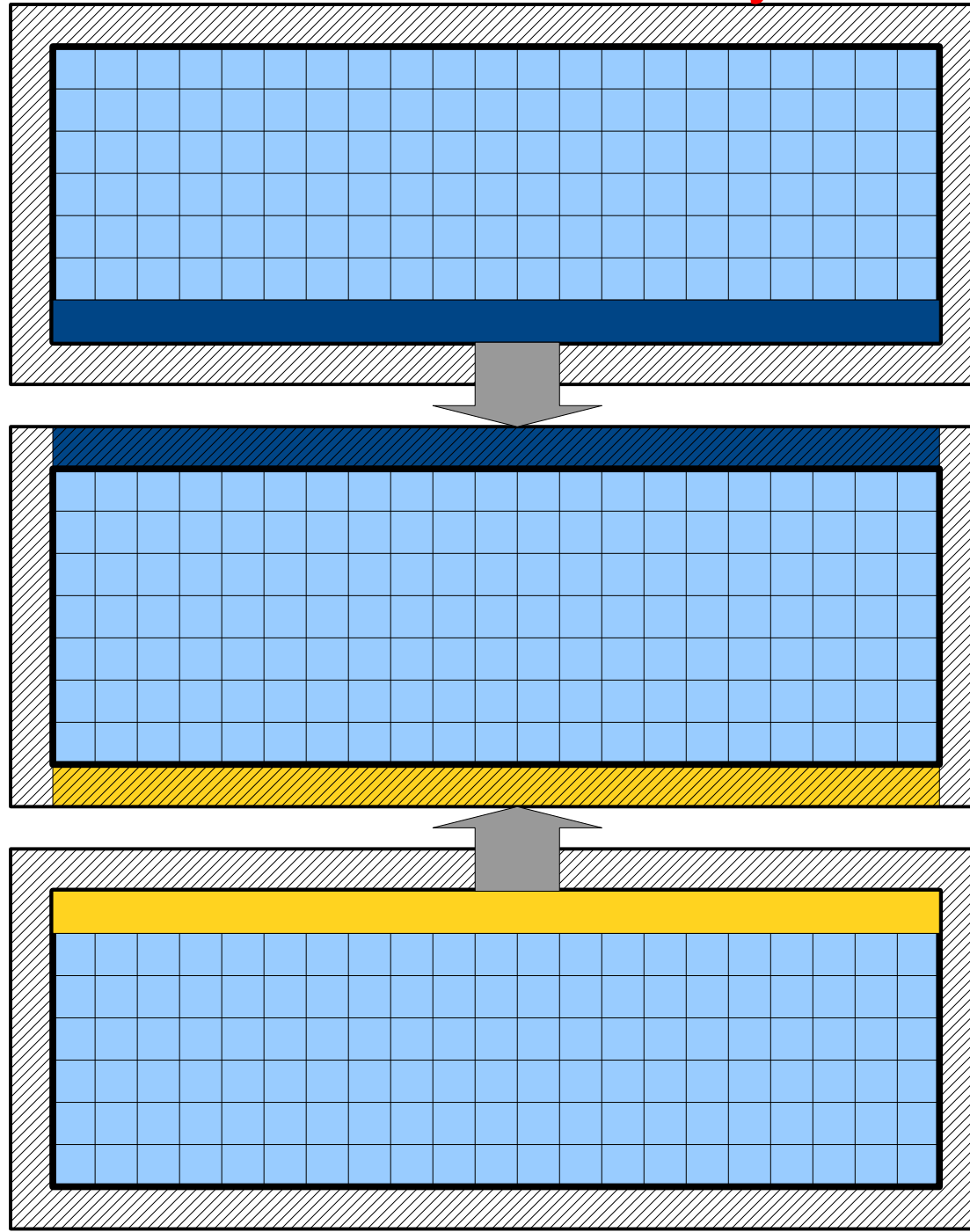
Example: 2D (Block, *) partitioning with 5P stencil Periodic boundary



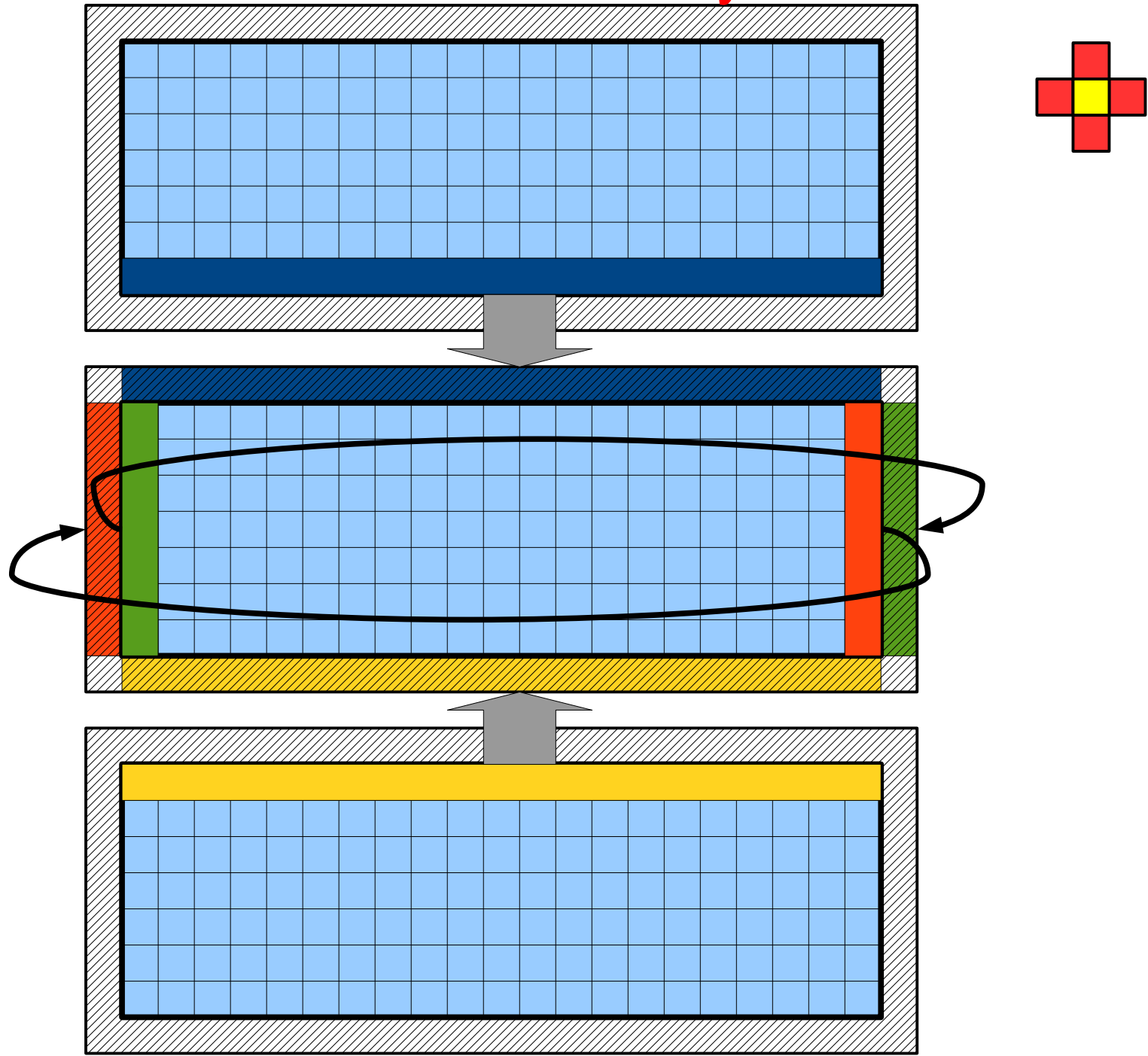
Example: 2D (Block, *) partitioning with 5P stencil Periodic boundary



Example: 2D (Block, *) partitioning with 5P stencil Periodic boundary



Example: 2D (Block, *) partitioning with 5P stencil Periodic boundary



2D Stencil Example: Game of Life

- 2D cyclic domain, each cell has two possible states
 - 0 = dead
 - 1 = alive
- The state of a cell at time $t + 1$ depends on
 - the state of that cell at time t
 - the number of alive cells at time t among the 8 neighbors
- Rules:
 - **Alive** cell with less than 2 alive neighbors → **dies**
 - **Alive** cell with two or three alive neighbors → **lives**
 - **Alive** cell with more than three alive neighbors → **dies**
 - **Dead** cell with three alive neighbors → **lives**

Example: Game of Life

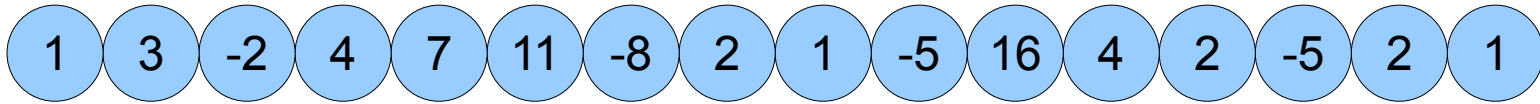
- See [game-of-life.c](#)

Reduce

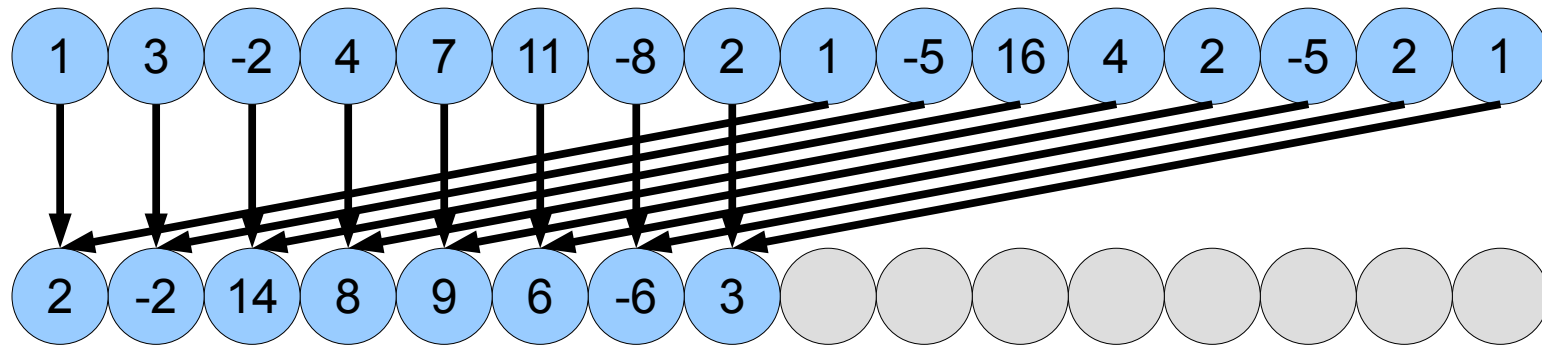
Reduce

- A **reduction** is the application of an **associative binary operator** (e.g., sum, product, min, max...) to the elements of an array $[x_0, x_1, \dots x_{n-1}]$
 - $\text{sum-reduce}([x_0, x_1, \dots x_{n-1}]) = x_0 + x_1 + \dots + x_{n-1}$
 - $\text{min-reduce}([x_0, x_1, \dots x_{n-1}]) = \min \{ x_0, x_1, \dots x_{n-1} \}$
 - ...
- A reduction can be realized in **$O(\log_2 n)$ parallel steps**

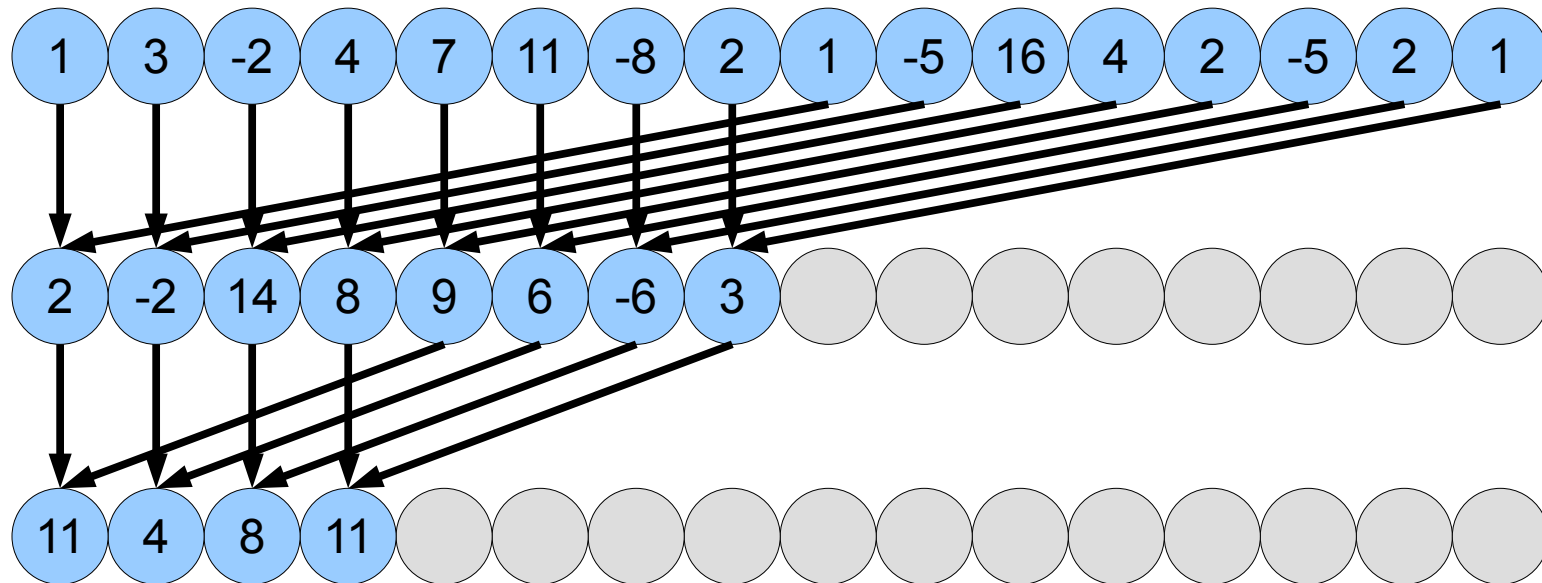
Example: sum-reduce



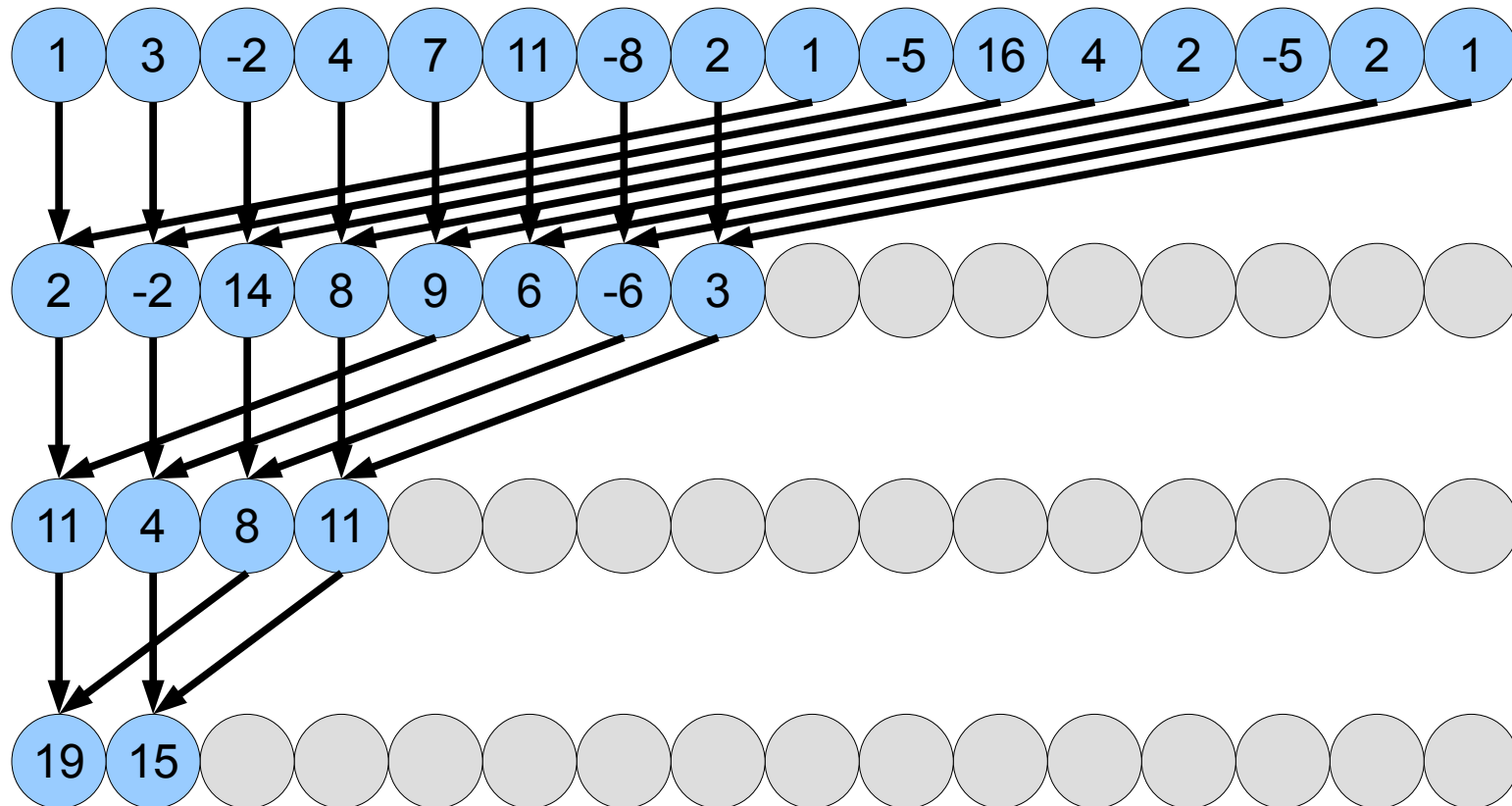
Example: sum-reduce



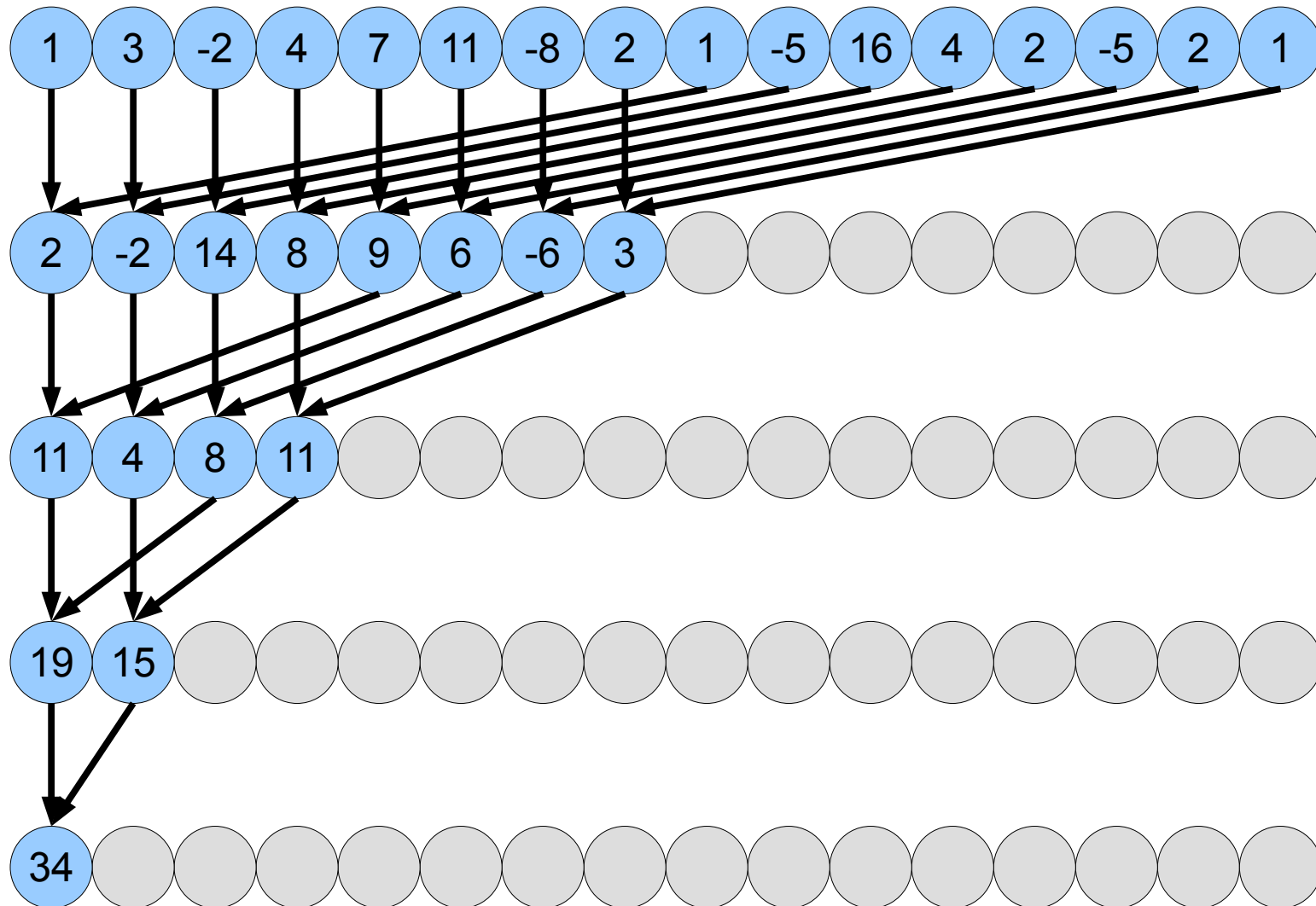
Example: sum-reduce



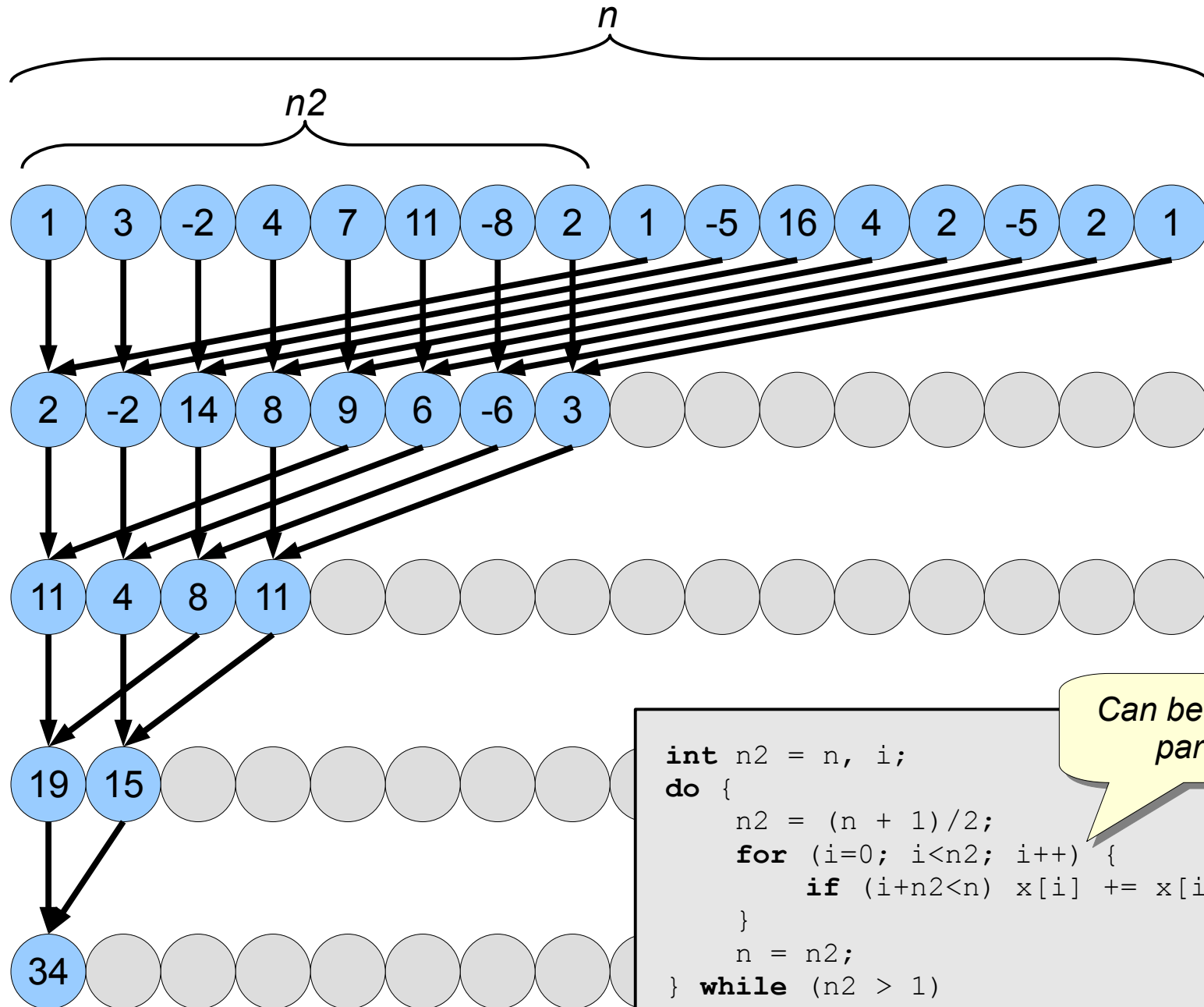
Example: sum-reduce



Example: sum-reduce



Example: sum-reduce



Parallel Program

```
int n2 = n, i;
do {
    n2 = (n + 1)/2;
    for (i=0; i<n2; i++) {
        if (i+n2<n) x[i] += x[i+n2];
    }
    n = n2;
} while (n2 > 1)
return x[0];
```

Can be done in parallel

See [reduction.c](#)

Scan

Scan (Prefix Sum)

- A **scan** computes all prefixes of an array $[x_0, x_1, \dots, x_{n-1}]$ using a given associative binary operator op (e.g., sum, product, min, max...)

$$[y_0, y_1, \dots, y_{n-1}] = \text{inclusive-scan}(op, [x_0, x_1, \dots, x_{n-1}])$$

where

$$y_0 = x_0$$

$$y_1 = x_0 \ op \ x_1$$

$$y_2 = x_0 \ op \ x_1 \ op \ x_2$$

...

$$y_{n-1} = x_0 \ op \ x_1 \ op \ \dots \ op \ x_{n-1}$$

Scan (Prefix Sum)

- A **scan** computes all prefixes of an array $[x_0, x_1, \dots, x_{n-1}]$ using a given associative binary operator op (e.g., sum, product, min, max...)

$$[y_0, y_1, \dots, y_{n-1}] = \text{exclusive-scan}(op, [x_0, x_1, \dots, x_{n-1}])$$

where

$$y_0 = 0$$

$$y_1 = x_0$$

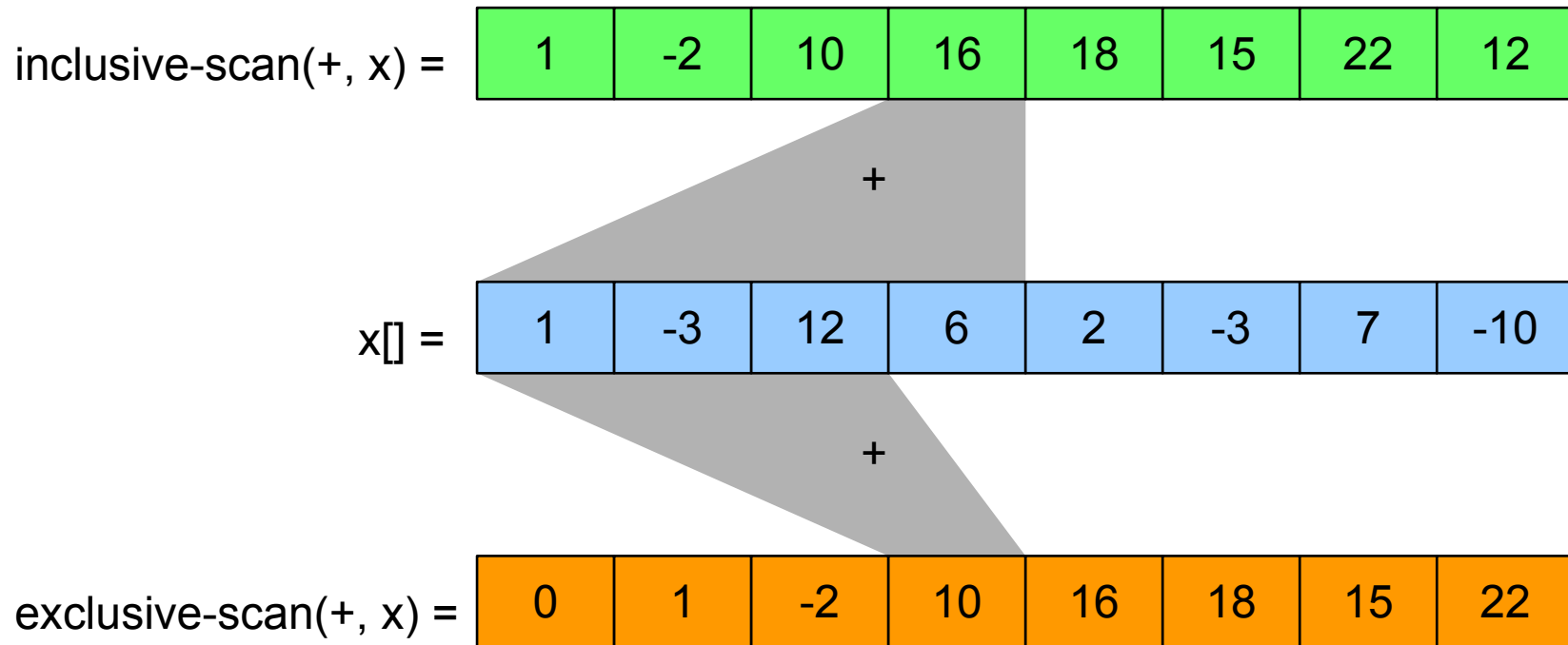
$$y_2 = x_0 \ op \ x_1$$

...

$$y_{n-1} = x_0 \ op \ x_1 \ op \ \dots \ op \ x_{n-2}$$

this is the neutral element of operator "op" (zero for sum, 1 for product, ...)

Example

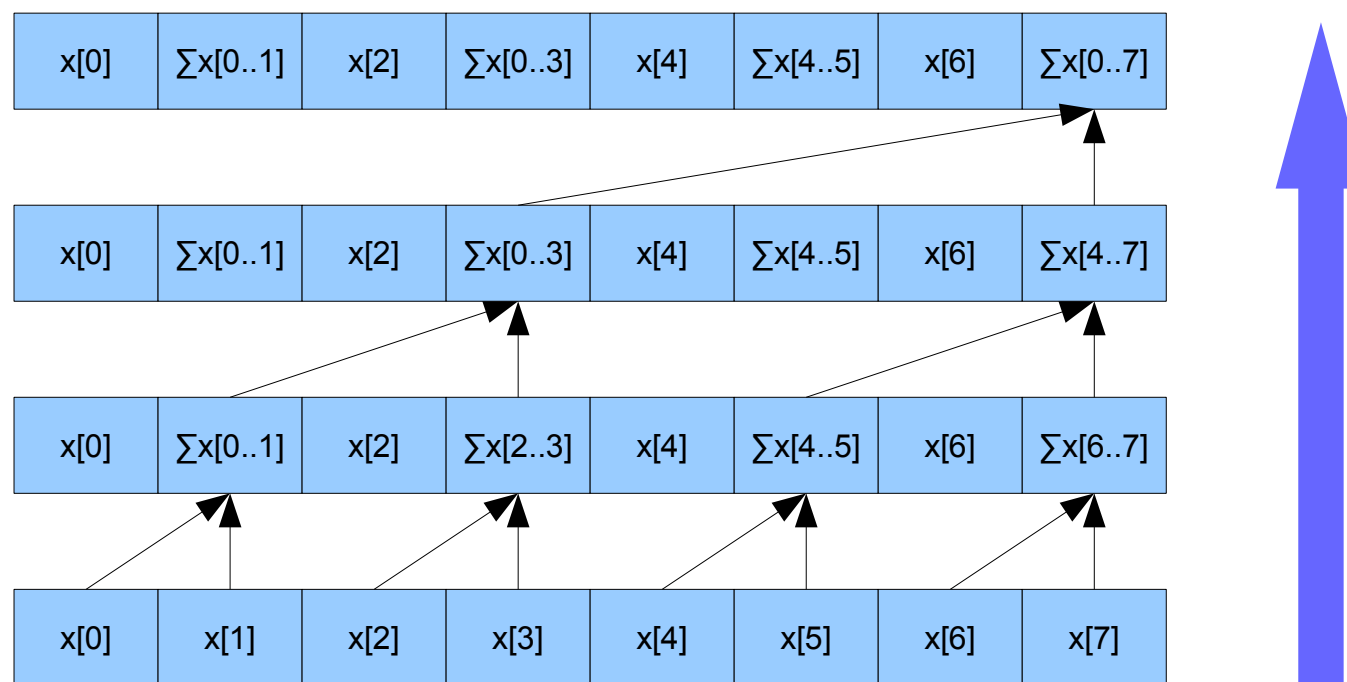


Serial implementation

```
void inclusive_scan(int *x, int *s, int n) // n must be > 0
{
    int i;
    s[0] = x[0];
    for (i=1; i<n; i++) {
        s[i] = s[i-1] + x[i];
    }
}

void exclusive_scan(int *x, int *s, int n) // n must be > 0
{
    int i;
    s[0] = 0;
    for (i=1; i<n; i++) {
        s[i] = s[i-1] + x[i-1];
    }
}
```


Exclusive scan: Up-sweep

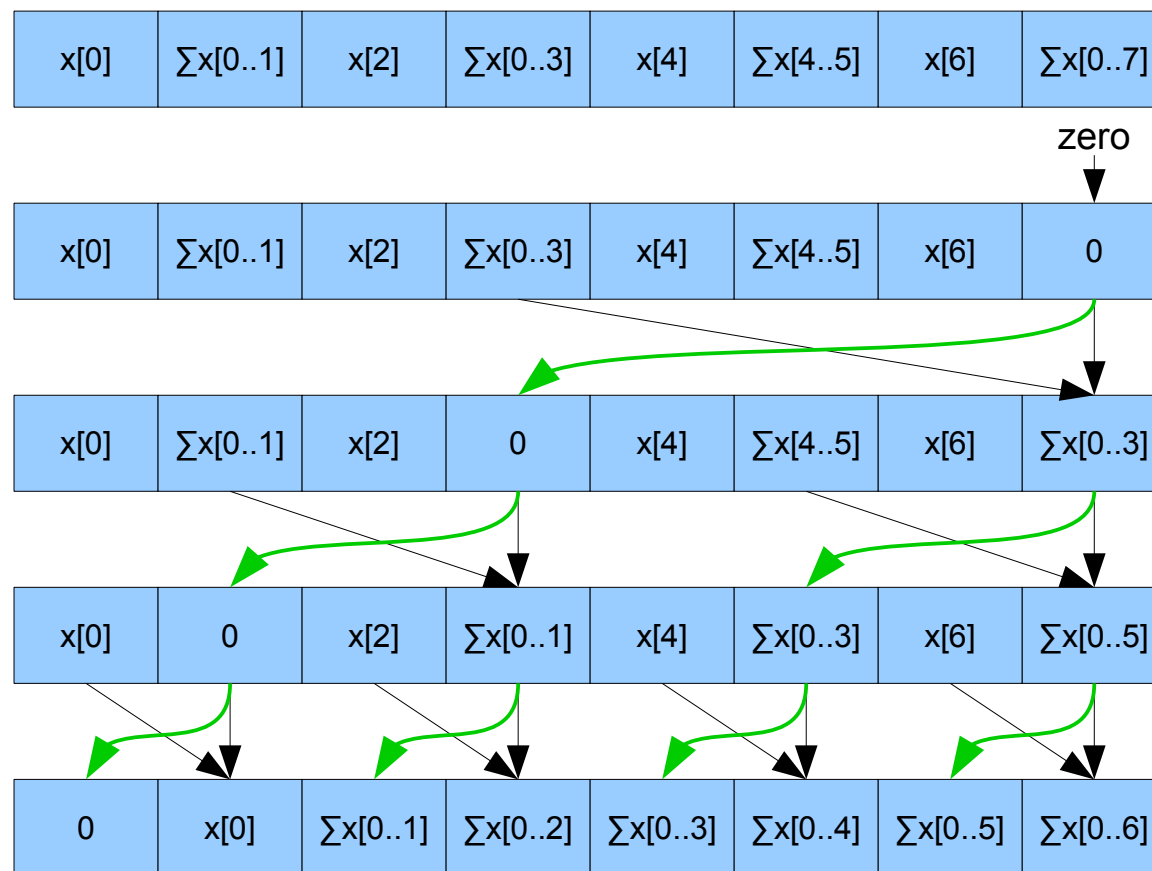


```
for ( d=1; d<n/2; d *= 2 ) {  
    for ( k=0; k<n; k+=2*d ) {  
        x[k+2*d-1] = x[k+d-1] + x[k+2*d-1];  
    }  
}
```

Can be done in parallel

$O(n)$ additions

Exclusive scan: Down-sweep



```

x[n-1] = 0;
for ( ; d > 0; d >>= 1 ) {
    for (k=0; k<n; k += 2*d ) {
        float t = x[k+d-1];
        x[k+d-1] = x[k+2*d-1];
        x[k+2*d-1] = t + x[k+2*d-1];
    }
}

```

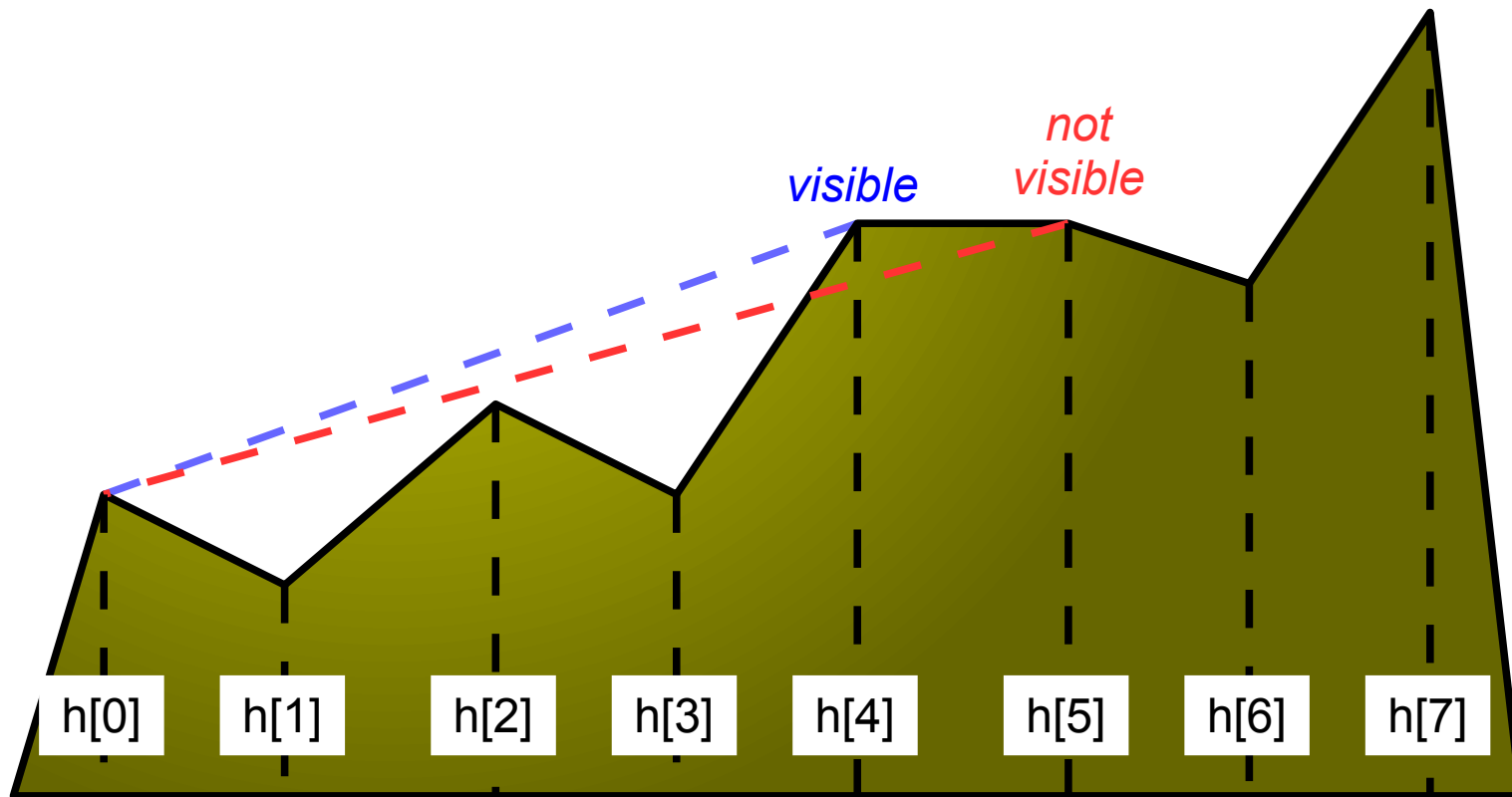
Can be done in parallel

See [prefix-sum.c](#)

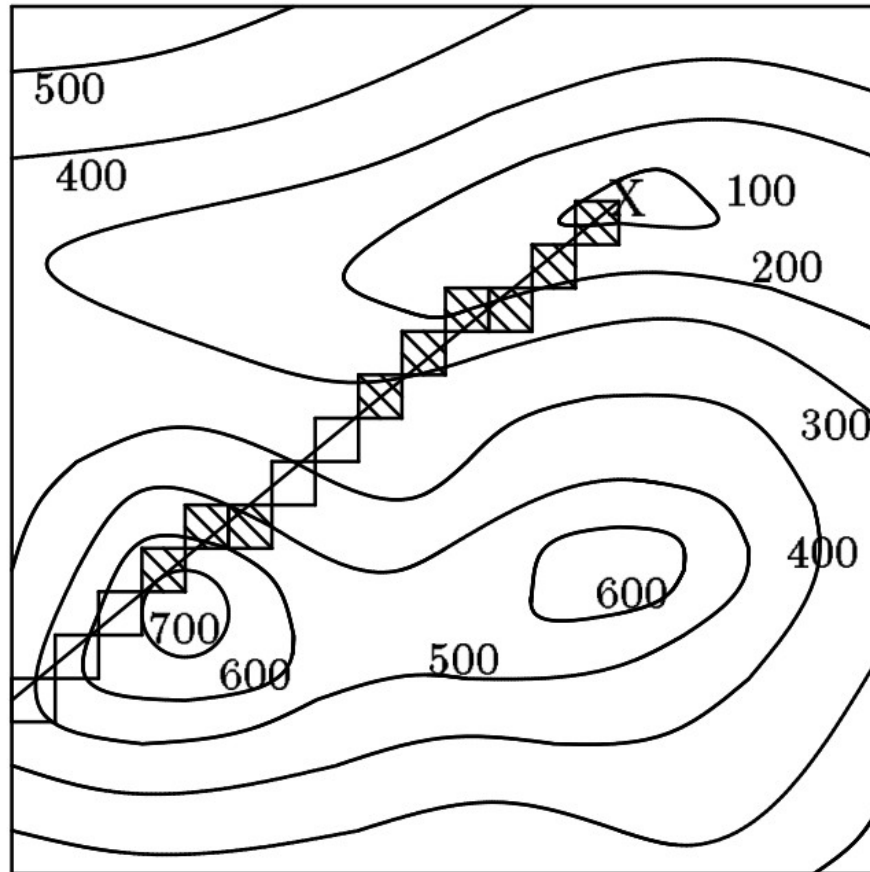
Application of the *scan* pattern

Example: Line of Sight

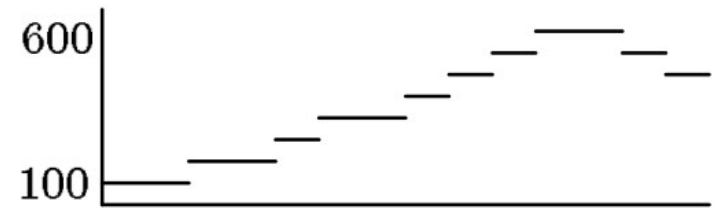
- n peaks of heights $h[0], \dots, h[n - 1]$; the distance between consecutive peaks is one.
- Which peaks are visible from peak 0?



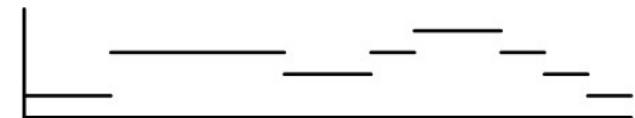
Line of sight



Altitude Map



Altitude Vector



Angle Vector

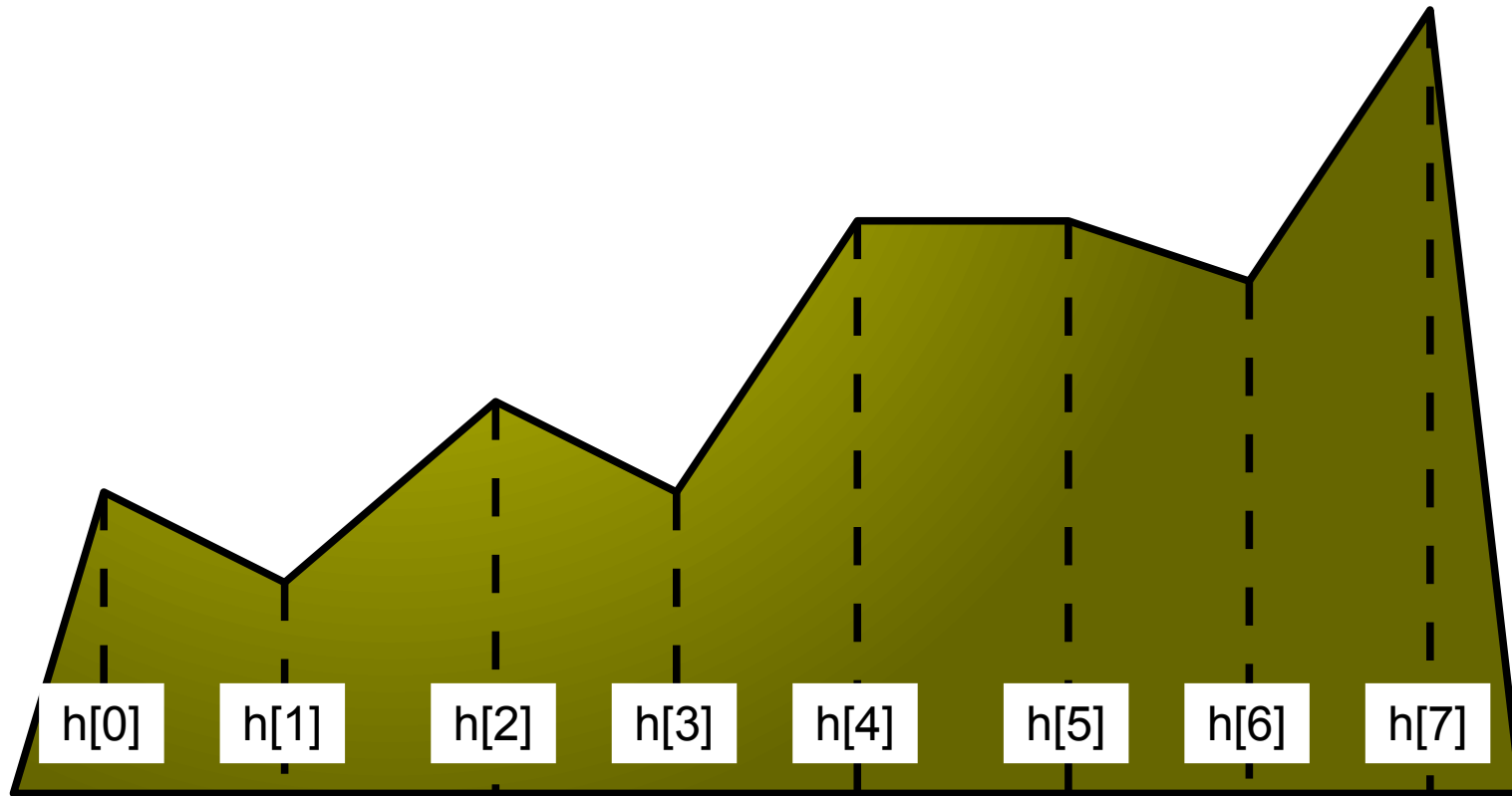


Max-Scan of Angle Vector

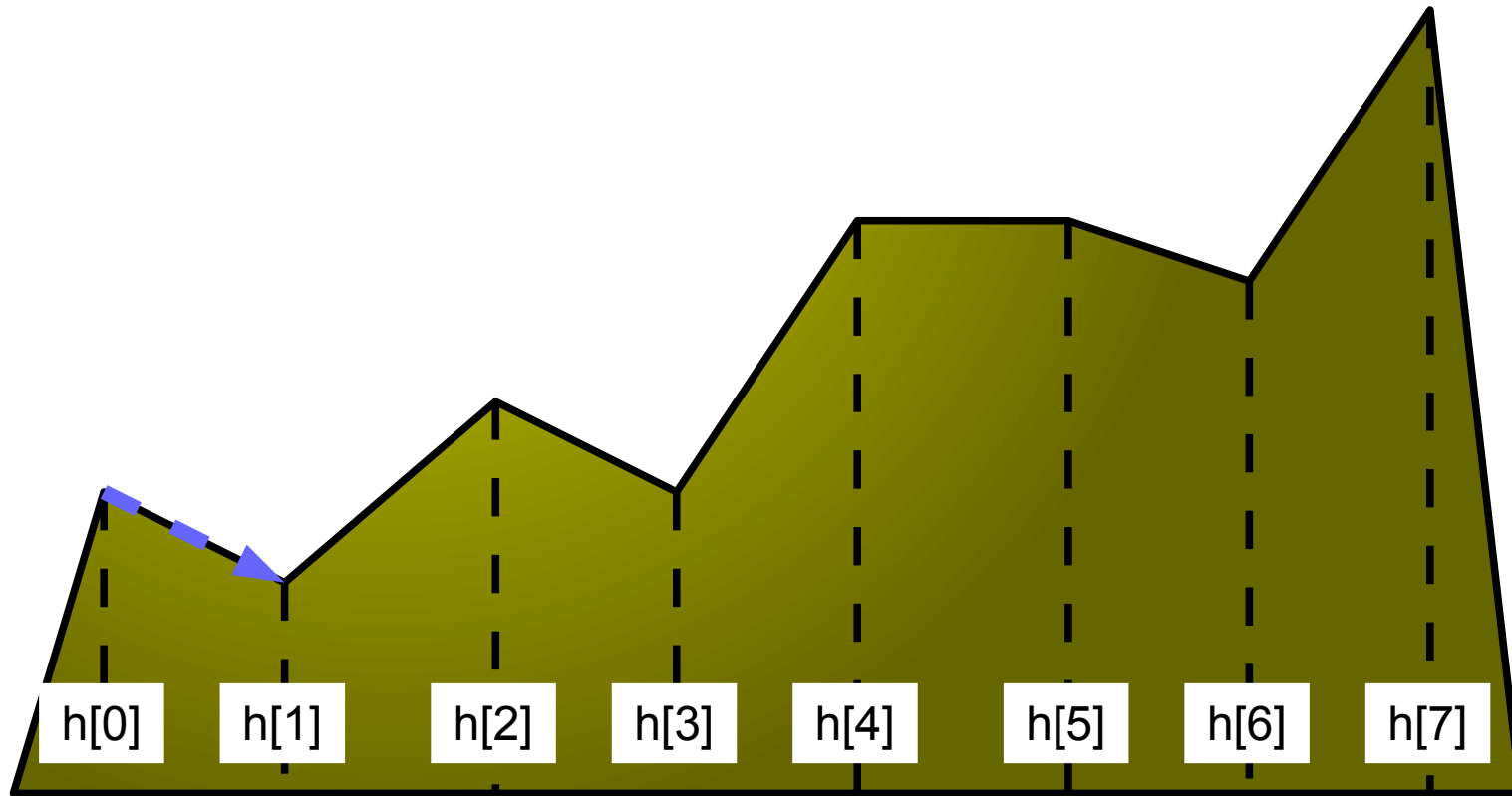
Ray Vectors

Source: Guy E. Blelloch, [Prefix Sums and Their Applications](#)

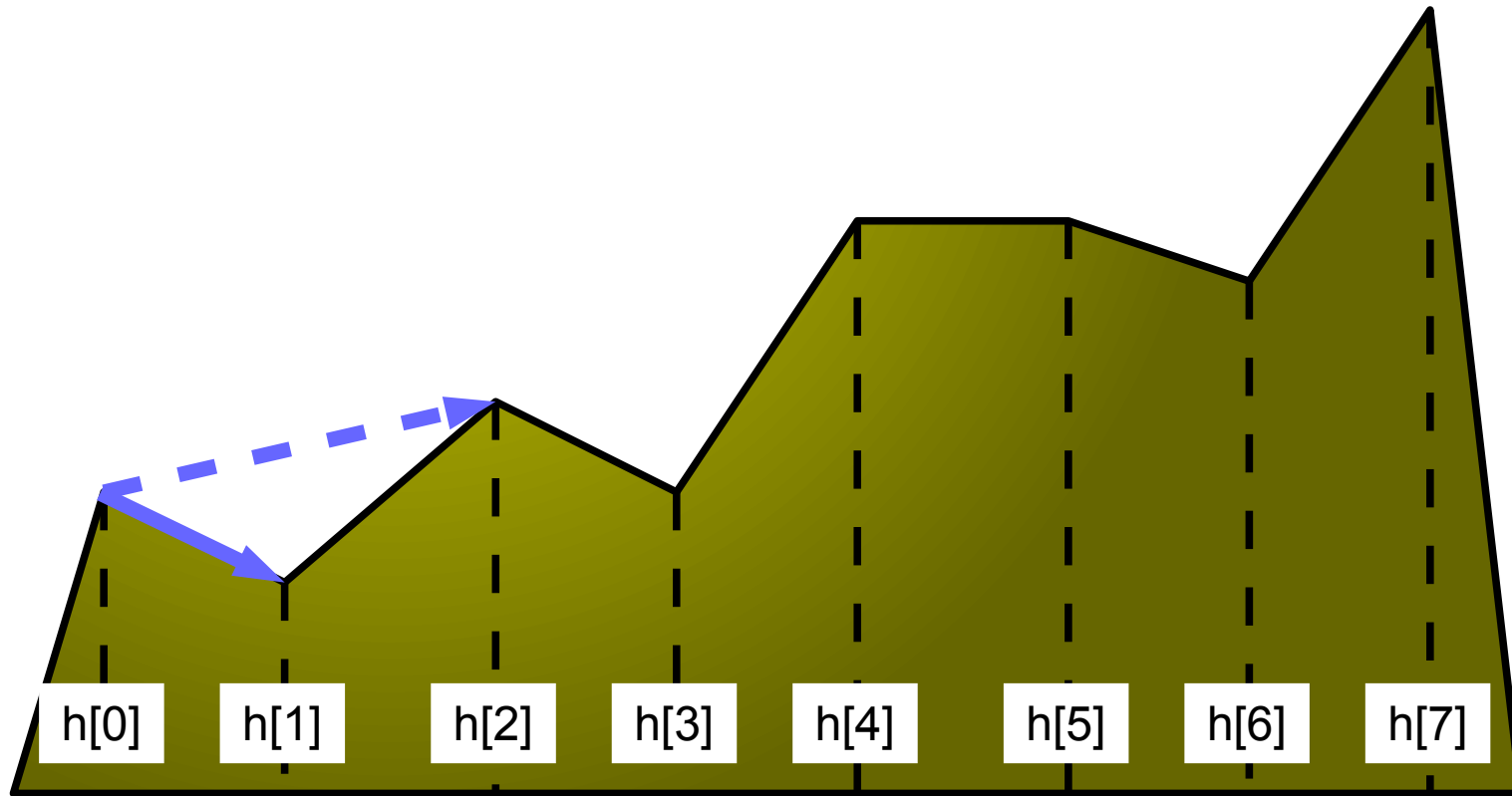
Line of sight



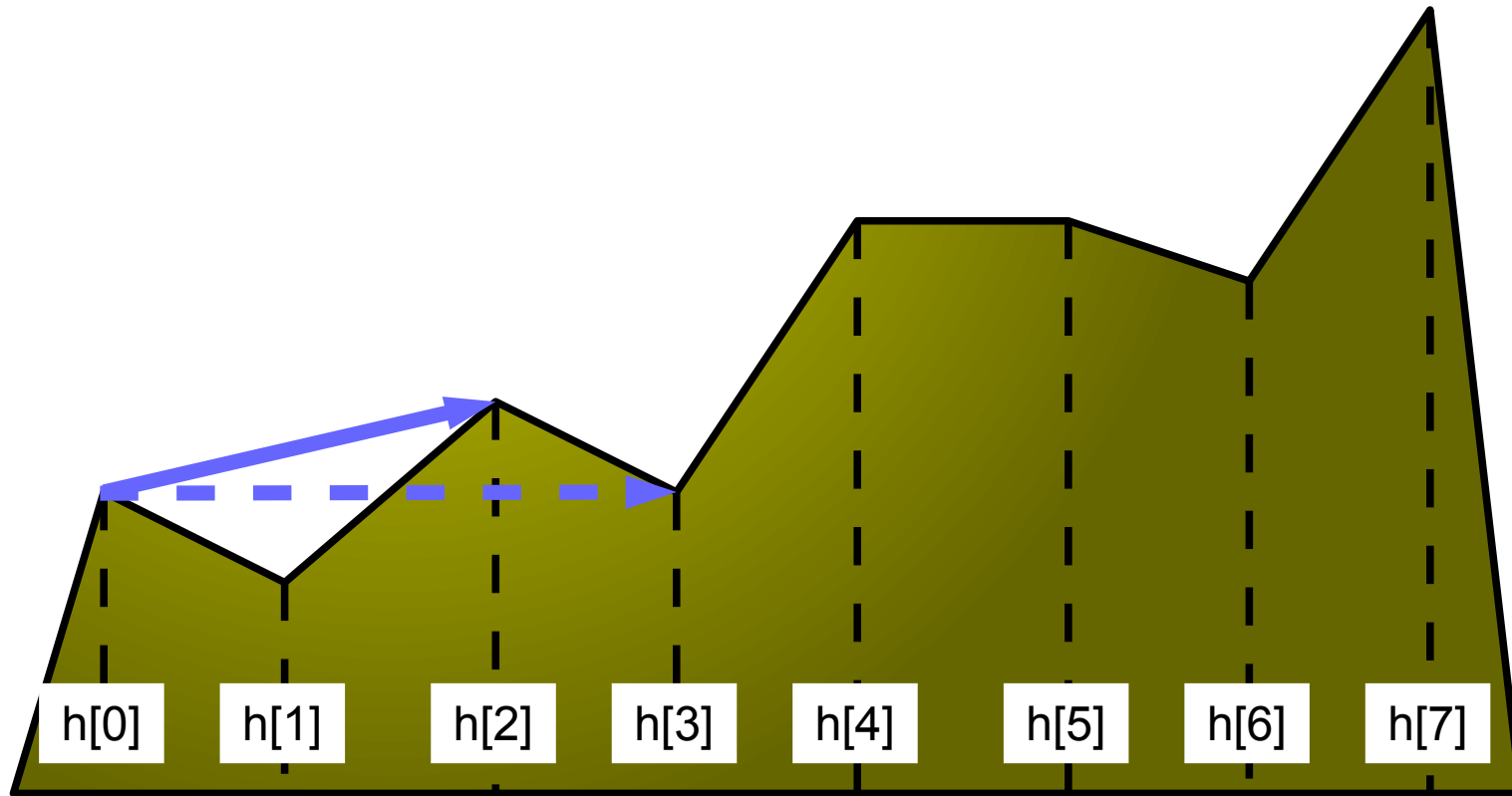
Line of sight



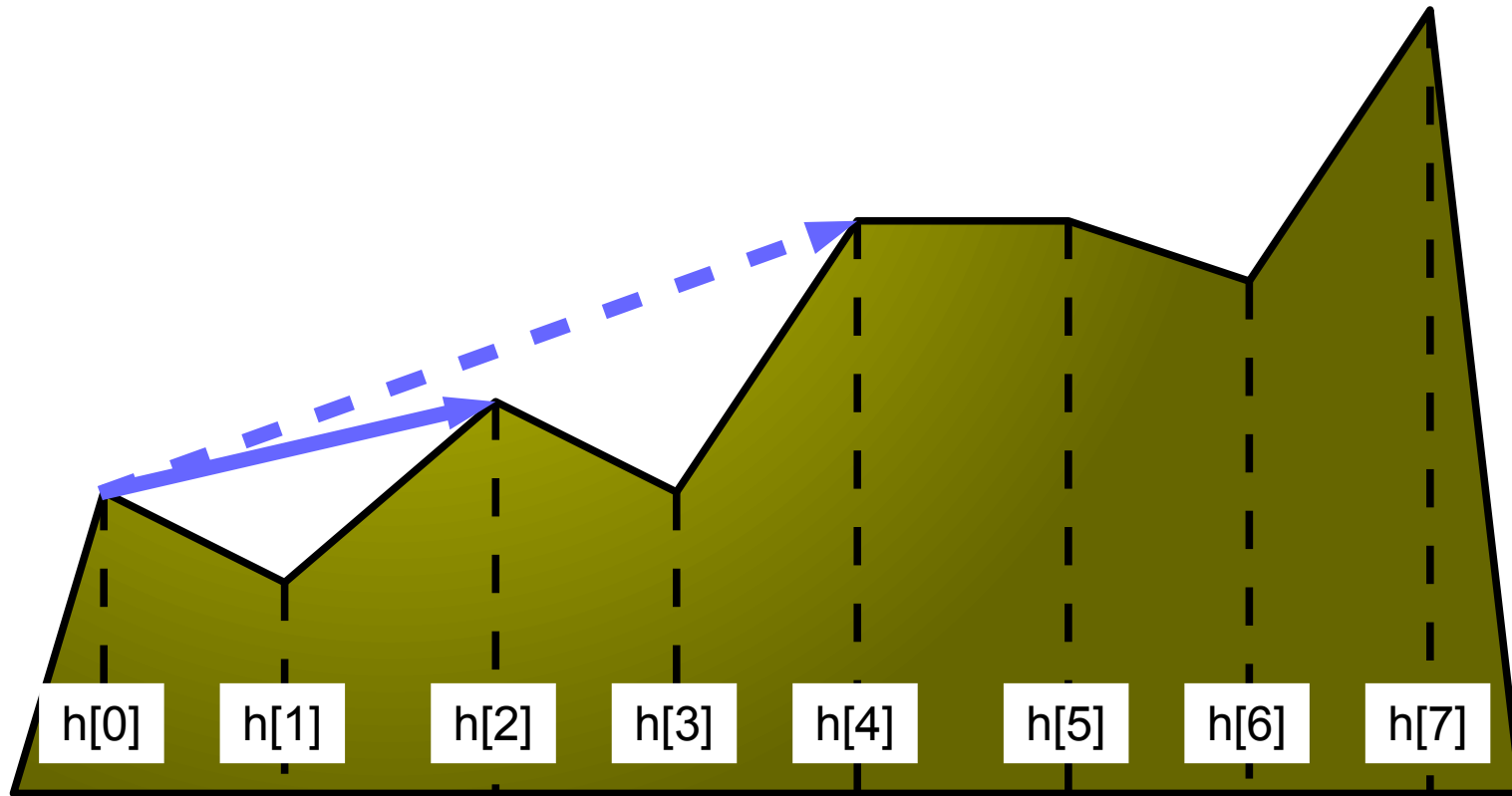
Line of sight



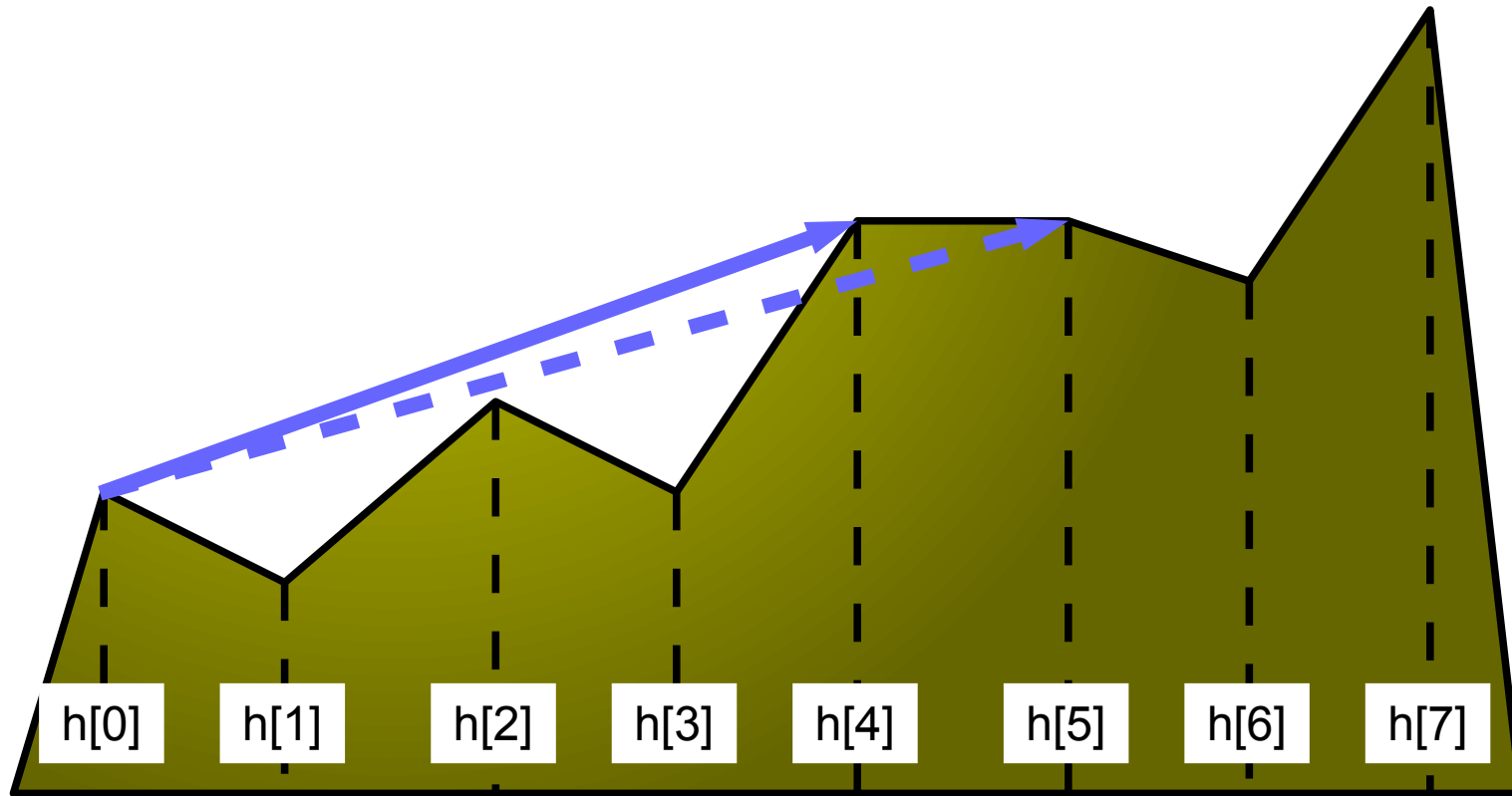
Line of sight



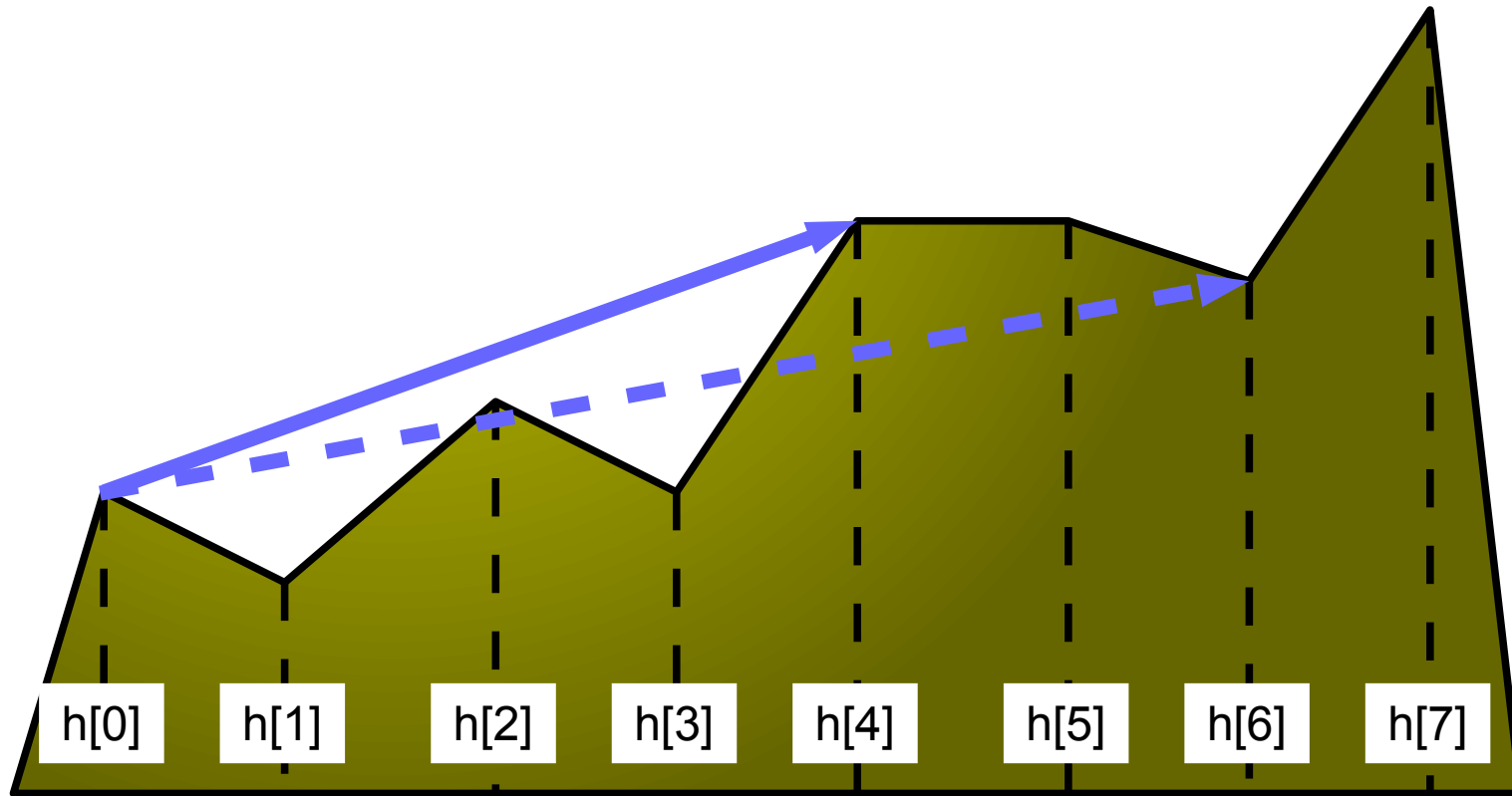
Line of sight



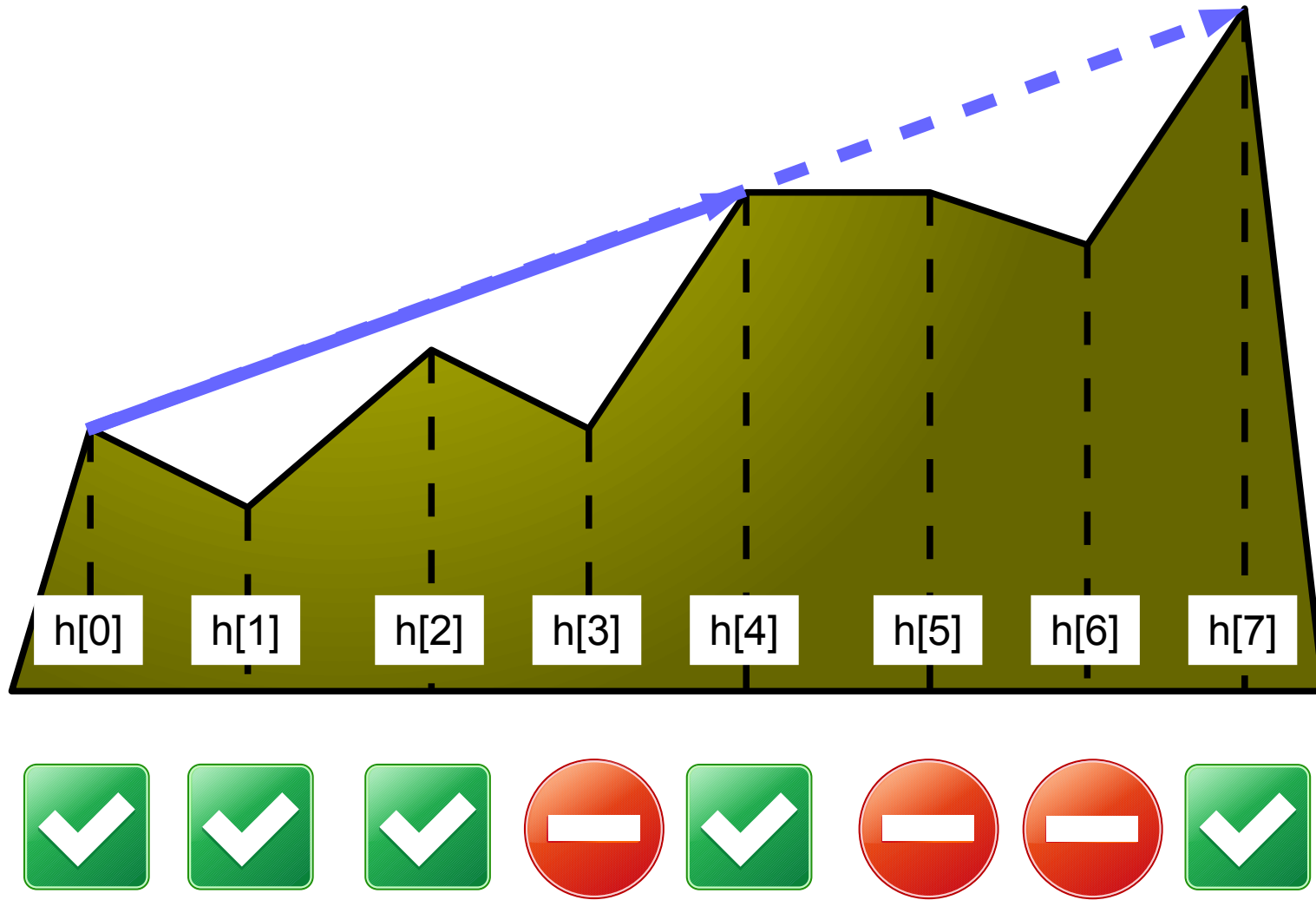
Line of sight



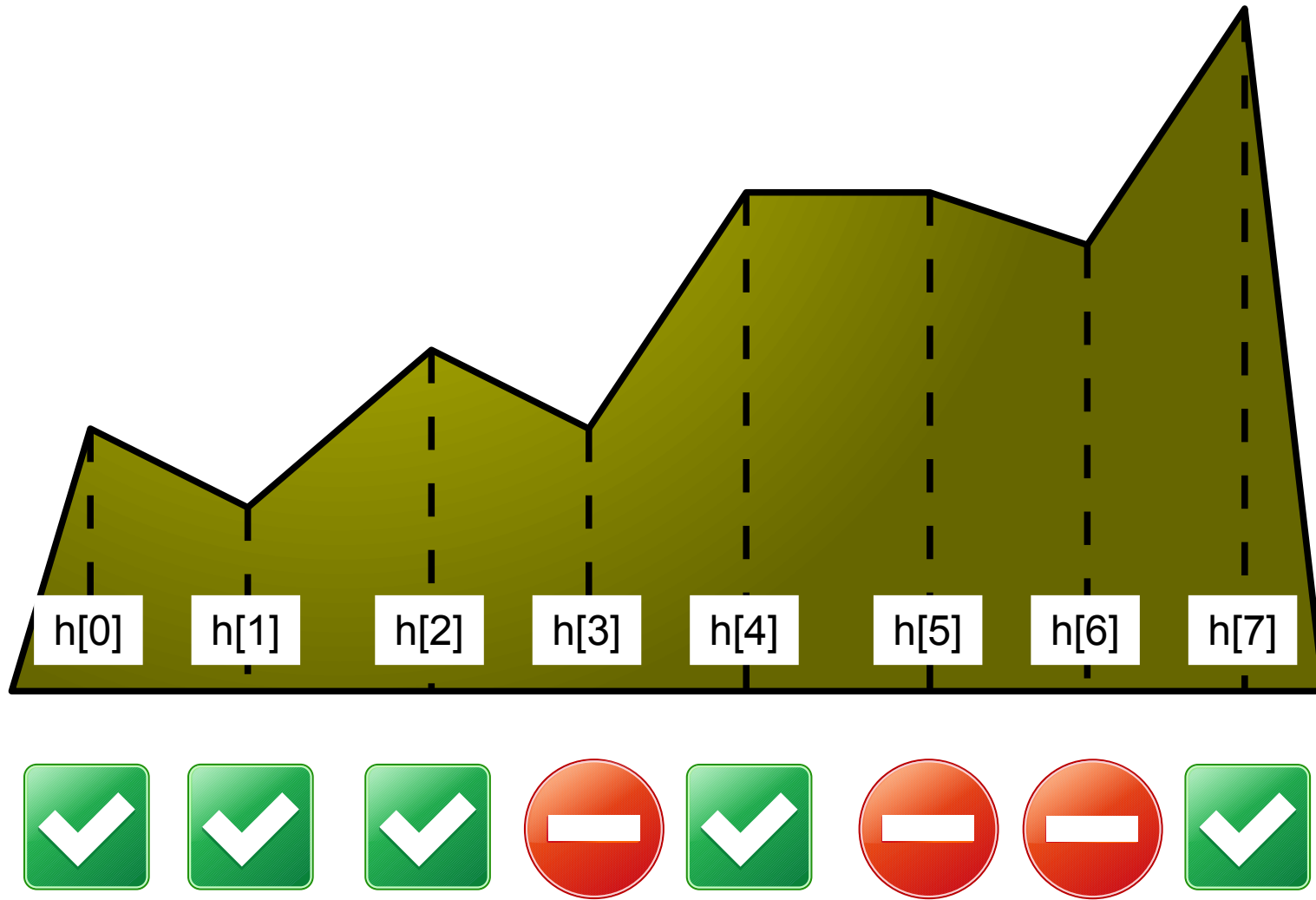
Line of sight



Line of sight



Line of sight



Serial algorithm

- For each $i = 0, \dots, n - 1$
 - Let $a[i]$ be the slope of the line connecting the peak 0 to the peak i
 - $a[0] \leftarrow -\infty$
 - $a[i] \leftarrow \arctan((h[i] - h[0]) / i),$ se $i > 0$
- For each $i = 0, \dots, n - 1$
 - $amax[0] \leftarrow -\infty$
 - $amax[i] \leftarrow \max \{a[0], a[1], \dots, a[i - 1]\},$ se $i > 0$
- For each $i = 0, \dots, n - 1$
 - If $a[i] \geq amax[i]$ then the peak i is visible
 - otherwise the peak i is not visible

Serial algorithm

```
bool[0..n-1] Line-of-sight( double h[0..n-1] )  
  bool v[0..n-1]  
  double a[0..n-1], amax[0..n-1]  
  a[0]  $\leftarrow -\infty$   
  for i  $\leftarrow$  1 to n-1 do  
    a[i]  $\leftarrow$  arctan( ( h[i] - h[0] ) / i )  
  endfor  
  amax[0]  $\leftarrow -\infty$   
  for i  $\leftarrow$  1 to n-1 do  
    amax[i]  $\leftarrow$  max{ a[i-1], amax[i-1] }  
  endfor  
  for i  $\leftarrow$  0 to n-1 do  
    v[i]  $\leftarrow$  ( a[i]  $\geq$  amax[i] )  
  endfor  
  return v
```


Serial algorithm

```
bool[0..n-1] Line-of-sight( double h[0..n-1] )  
  bool v[0..n-1]  
  double a[0..n-1], amax[0..n-1]  
  a[0]  $\leftarrow -\infty$   
  for i  $\leftarrow$  1 to n-1 do  
    a[i]  $\leftarrow$  arctan( ( h[i] - h[0] ) / i )  
  endfor  
  amax[0]  $\leftarrow -\infty$   
  for i  $\leftarrow$  1 to n-1 do  
    amax[i]  $\leftarrow$  max{ a[i-1], amax[i-1] }  
  endfor  
  for i  $\leftarrow$  0 to n-1 do  
    v[i]  $\leftarrow$  ( a[i]  $\geq$  amax[i] )  
  endfor  
  return v
```

Embarrassingly
parallel

Embarrassingly
parallel

Parallel algorithm

```
bool[0..n-1] Parallel-line-of-sight( double h[0..n-1] )
    bool v[0..n-1]
    double a[0..n-1], amax[0..n-1]
    a[0] ← -∞
    for i ← 1 to n-1 do in parallel
        a[i] ← arctan( ( h[i] - h[0] ) / i )
    endfor

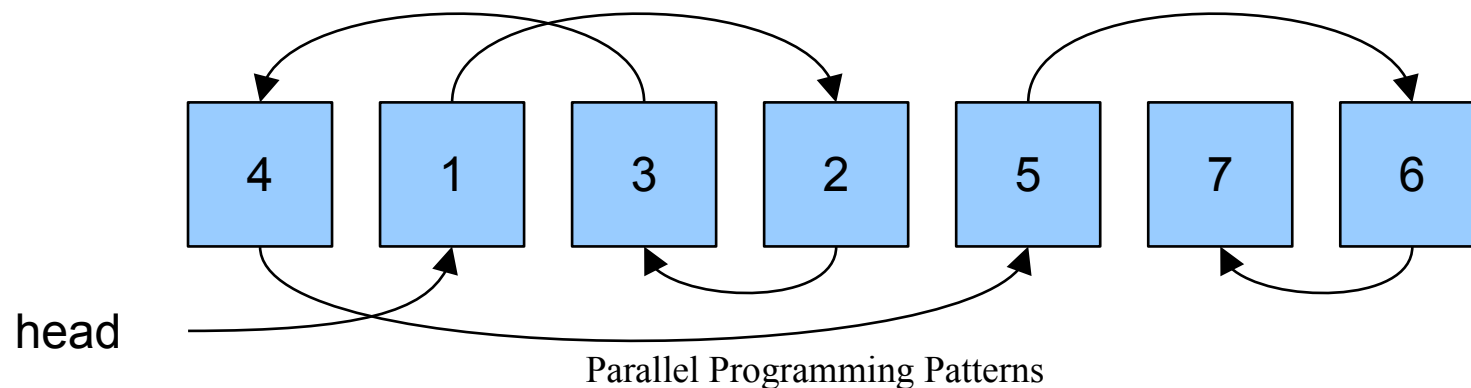
    amax ← exclusive-scan( max, a )

    for i ← 0 to n-1 do in parallel
        v[i] ← ( a[i] ≥ amax[i] )
    endfor
    return v
```

List Ranking (Pointer Jumping)

Problem formulation

- Input: a linked list.
- Output: label each node with its position in the list.
 - The first node has position 1.
 - Its successor has position 2.
 - ...
- Many useful applications:
 - Connected components labeling, spanning trees, ...



Serial solution

```
typedef struct list_node {
    int rank;
    struct list_node *next;
} list_node;

void rank( list_node *item )
{
    int r = 0;
    while (item != NULL) {
        item->rank = r++;
        item = item->next;
    }
}
```

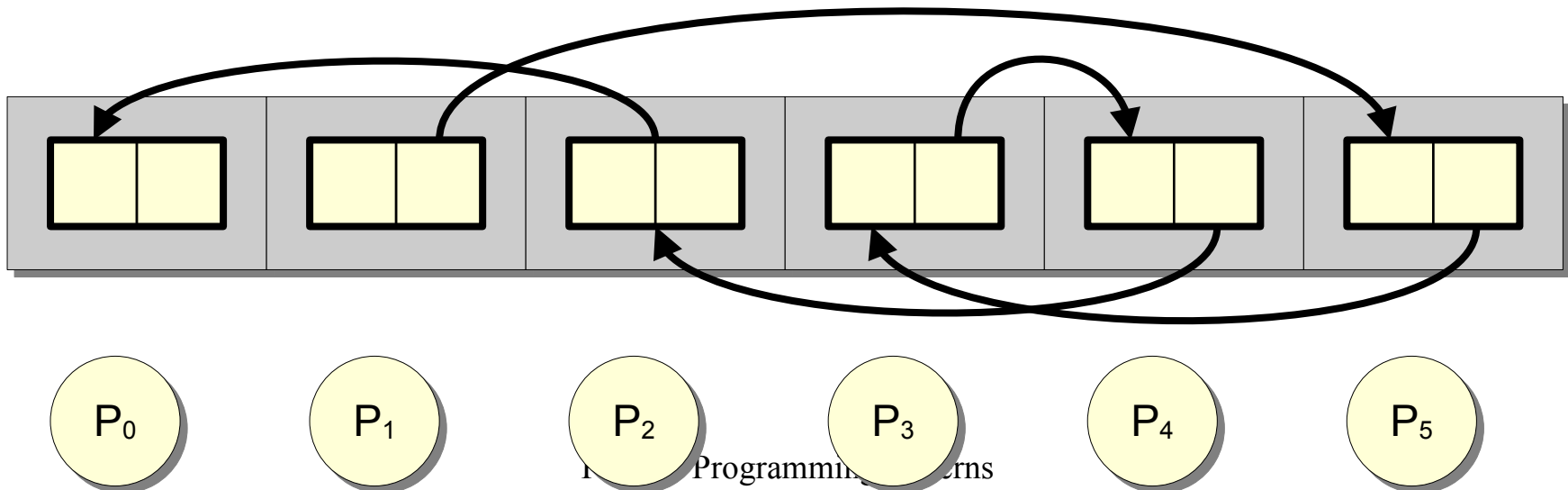
- The serial solution appears to be intrinsically serial due to a loop-carried dependence on **item**.
 - Stay tuned...

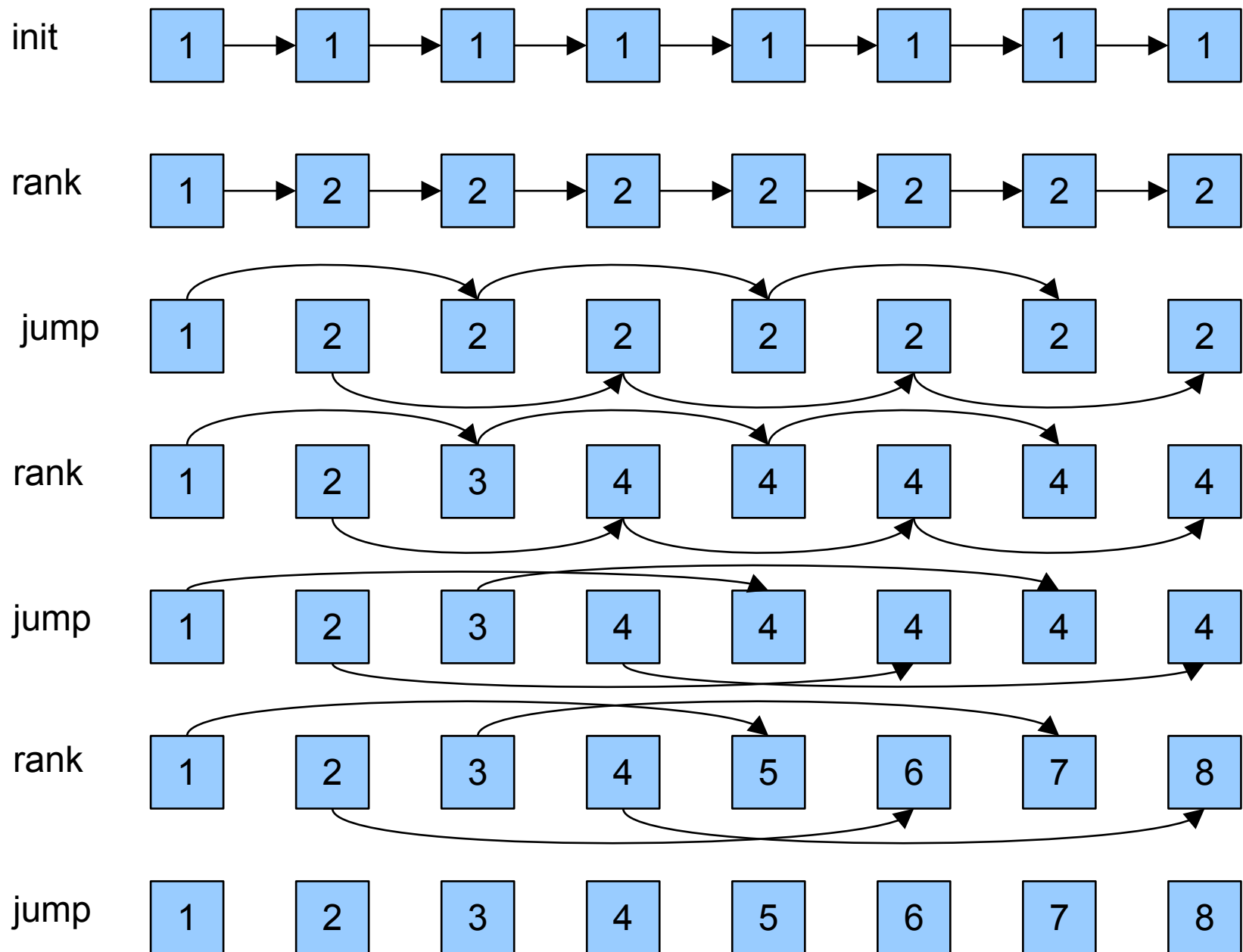
Pointer Jumping

- One processor per list node.
- Init:
 - Each processor sets `item->rank = 1`
- Main loop to be executed synchronously by all processors
 - `if (item->next != NULL)`
`item->next->rank += item->rank` (*rank*)
 - All processors read `item->next->rank` at the same time, and store the value in a local register.
 - All processors read `item->rank` at the same time, and store the value in a local register.
 - All processors update `item->next->rank` at the same time.
 - `if (item->next != NULL)`
`item->next = item->next->next` (*jump*)
 - All processors read `item->next->next` at the same time, and store the value in some local register.
 - All processors set `item->next` to that value at the same time.

Data representation

- We use n processors to rank a list of n nodes.
- Each processor is associated to a node.
- To make the association easier, the list nodes are stored in an array.
 - However, they do *not* need to be stored in rank order, otherwise the problem would be trivial!





Span

- If the list has n nodes, the main loop requires $O(\log n)$ iterations.
 - The first *jump* splits the list into two sublists.
 - Subsequent *jump* split the lists into two halves.
 - After at most $O(\log n)$ *jump*, lists have a single element.
- Each iteration can be executed in parallel by $O(n)$ processors
 - Even if we relax the synchronous execution requirement.
 - We will see in the lab...

Conclusions

- A parallel programming patterns defines:
 - a partitioning of the input data
 - a communication structure among parallel tasks
- Parallel programming patterns can help to define efficient algorithms
 - Many problems can be solved using one or more known patterns

Bibliography

- Timothy G. Mattson, Beverly A. Sanders, Berna L. Massingill, *Patterns for Parallel Programming*. Addison-Wesley. 2025, ISBN 0-321-22811-1.
- Michael McCool, Arch D. Robison, James Reinders, *Structured Parallel Programming*, Morgan Kaufmann, 2012, ISBN 978-0-12-415993-8.

